

第七章 虚拟存储管理

提纲

7.1 外存空间管理

7.2 虚拟存储系统

提纲

7.1 外存空间管理

7.2 虚拟存储系统

7.1 外存资源管理

- 外存空间

Swap空间	File空间	输入井	输出井
--------	--------	-----	-----

7.1 外存资源管理

- 外存空间划分
 - 静态等长， 2^i ，称为一块(block)，块是外存分配的基本单位，也是IO传输的基本单位
 - 如果需求不足一个完整块，会形成块内“零头”。由于外存空间很大，块内零头忽略
- 外存空间分配
 - 空闲块链(慢)：所有空闲块连成一个链
 - 空闲块表：相连的空闲块记录在同一项中
 - 字位映像图：用1位表示一个块的状态

进程与外存对应关系

- 界地址
 - 每进程占一组外存连续块
 - 每进程占二组外存连续块(双对界)
- 页式
 - 内存一页，外存一块。内存页与外存块长度相同
- 段式
 - 每段占外存若干连续块，进程的多个段之间在外存可以不连续
- 段页式
 - 内存一页，外存一块。内存页与外存块长度相同。段内多页所对应的外存块可以不连续，一个进程的多段可以分散在外存的不同区域中

7.2 虚拟存储系统

- 无虚拟问题
 - 不能运行比内存大的程序
 - 进程全部装入内存，浪费空间（进程活动具有局部性）
 - 进程在一个有限的时间段内访问的代码和数据往往集中在有限的地址范围内
 - 把程序一部分装入内存存在较大概率上也足够让其运行一小段时间
- 虚拟存储
 - 在程序运行时，只将当前必要的很小一部分代码和数据装入内存，其余装入外存，运行时访问在外存部分动态调入，内存不够淘汰

7.2 虚拟存储系统（Cont.）

- 虚拟存储目标
 - 使得大的程序能在较小的内存中运行
 - 使得多个程序能在较小的内存中运行
 - 使得多个程序并发运行时地址不冲突
 - 使得内存利用效率高
- 虚拟存储管理
 - 虚拟页式
 - 虚拟段式
 - 虚拟段页式

7.2.1 虚拟页式存储系统

■ 基本原理

■ 进程运行前

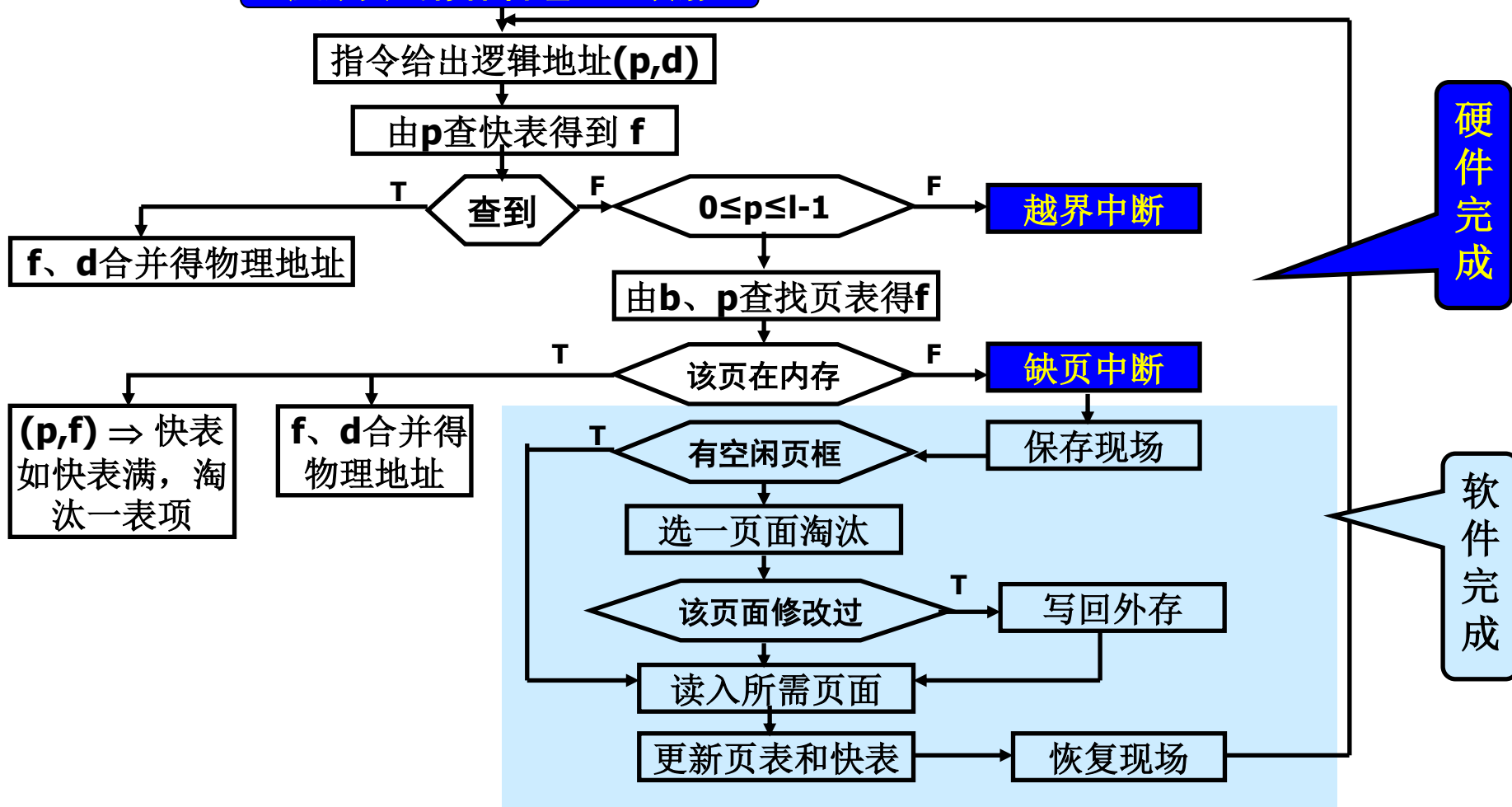
- 部分页面装入内存，另一部分页面(或全部页面)装入外存

■ 进程运行时

- 访问页在内存，与无虚拟情形相同
- 访问页不在内存，发生缺页中断，中断处理程序
 - 找到访问页在外存的地址
 - 在内存找一空闲页面
 - 如没有，按淘汰算法淘汰一个
 - 如需要，将淘汰页面写回外存，修改页表和总页表
 - 读入所需页面，修改页表及快表（切换进程）
 - 重新启动中断前指令

7.2.1 虚拟页式存储管理(Cont.)

虚拟页式存储管理地址映射



7.2.1 虚拟页式存储管理(Cont.)

- 缺页中断

- 定义

- 在地址映射过程中，当需要访问的页不在内存时，则系统产生缺页中断

- 缺页中断处理程序

- 将所缺的页从外存调度内存的某个页框中，更新页表和快表

对页表的改进：

逻辑页号	页框号	外存块号	内外标识	访问权限	修改标志
...	...	...	...	...	...
p	f	b'	(0,1)	{r,w,e}	(0,1)
...	...	...	...	...	...

对快表的改进：

逻辑页号	页框号	访问权限	修改标志
...	...	...	...
p	f	{r,w,e}	(0,1)
...	...	...	...

7.2.1.2 内存页框分配策略(静态策略)

在内存容量和进程数量确定的前提下，内存分配方法如下：

1. **平均分配**：将内存中的所有物理页框等分给进入系统中的进程。如内存128页，进程25个，每个进程5个页框
2. 按进程**长度**比例分配：按程序长度的比例确定分配内存的物理页框数。

p_i 共 s_i 个页面； $S = \sum s_i$ ；内存共 m 个页框

分配给进程 p_i 的物理页框数 $a_i = (s_i / S) \times m$

例：内存物理页框62，系统2个进程， p_1 逻辑页面127， p_2 逻辑页面10，则 p_1 分得57个物理页框， p_2 分得5个物理页框。

3. 按进程**优先级**比例分配：根据优先级别按比例分配内存物理页面。 r_i 表示进程 p_i 的优先级。

$S = \sum r_i$ ；内存共 m 个页框

分配给进程 p_i 的物理页框数 $a_i = (r_i / S) \times m$

4. 按进程**长度和优先级别**比例分配： p_i 共 s_i 个页面， r_i 表示进程 p_i 的优先级。进程 p_i 逻辑页面数与优先级之和为 $s_i + r_i$ 。

$S = \sum (r_i + s_i)$ ；内存共 m 个页框

分配给进程 p_i 的物理页框数 $a_i = ((s_i + r_i) / S) \times m$

以上4种方法都是静态的，静态策略没有反映：

(1) 程序结构； (2) 程序在不同时刻的行为特性。

7.2.1.3 外存块的分配策略

1. 静态分配

外存保持进程的全部页面，当某一外存页面被调入内存，所占用的外存页面并不释放

优点：速度快——淘汰时不必写回（未修改情况）

缺点：外存浪费

2. 动态分配

外存仅保持进程不在内存的页面，当某一外存页面被调入内存，将释放所占用的外存页面

优点：节省外存

缺点：速度慢——淘汰时必须写回

7.2.1.4 页面调入时机

1. 请调(demand paging)

upon page fault, 发生缺页中断时调入。

缺点：从缺页中断到所需页面被调入内存，期间对应的进程需要等待，将影响进程的推进速度。

2. 预调(prepaging)

before page fault, 将要访问时调入(根据程序顺序行为，不一定准)。预调可以节省因页故障而等待的时间。

当访问预先未调入内存的页面时，会发生缺页中断，因此预调必须辅以请调。

7.2.1.5 置换算法(replacement algorithm)

- 置换算法(淘汰算法)
 - 用于确定页面的调出原则
 - 被移出页面称为淘汰页面
 - 用于选择被淘汰页面的算法称为置换算法
- 用于：页淘汰、段淘汰、快表淘汰
- Objective: lowest page-fault rate
- 置换方式
 - 局部置换 全局置换
 - 页故障率: $f=F/A$ (F为故障次数, A为总访问次数)

7.2.1.5 置换算法(replacement algorithm)

1. 最佳淘汰算法(OPT--optimal)：淘汰以后不再需要的或者在最长时间以后才会用到的页面。

例：内存为该进程分配3个物理页框，开始为空。

6	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	6	0	1
6	6	6	2		2		2			2			2				6		
	0	0	0		0		4			0			0				0		
		1	1		3		3			3			1				1		
x	x	x	x		x		x			x			x				x		

页故障次数9

页故障率最低，为最理性的淘汰算法。然而，不能实现，
因为无法准确预期页面“将来”的被访问情况。

可以作为衡量其它算法性能的一个标准。

2. 先进先出 (FIFO)：淘汰最先调入内存的页面。

依据：先进入的可能已经使用完毕。

实现：采用队列的形式。

negative case：有些代码和数据可能整个程序运行中用到。

例子：1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

当分配内存3个和4个物理页框时，求页故障率

FIFO淘汰算法 (belady异常)

访问序列:	1	2	3	4	1	2	5	1	2	3	4	5
内存页面:	1	1	1	4	4	4	5			5	5	
		2	2	2	1	1	1			3	3	
			3	3	3	2	2			2	4	

页故障率=9/12

访问序列:	1	2	3	4	1	2	5	1	2	3	4	5
内存页面:	1	1	1	1			5	5	5	5	4	4
		2	2	2			2	1	1	1	1	5
			3	3			3	3	2	2	2	2
				4			4	4	4	3	3	3

页故障率=10/12

- Belady异常
 - 增加进程所分得的内存页框数不一定能够保证缺页故障次数下降
- Belady异常
 - 内存3个物理页面：页故障率=9/12
 - 内存4个物理页面：页故障率=10/12

3. 最近最少使用算法 (LRU)：淘汰最近一次访问时间距离当前时间间隔最长的页面。

实现：stack

当一页面被访问时，从栈中取出压到栈顶，
栈底是：LRU page.

例子：reference string: 4, 7, 0, 7, 1, 0, 2, 1, 7, 4
(页框数= 4)

				1	0	2	1	7	4
		0	7	7	1	0	2	1	7
	7	7	0	0	7	1	0	2	1
4	4	4	4	4	4	7	7	0	2

■ LRU算法

6	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	6	0	1
6	6	6	2		2		4	4	4	0			1		1		1		
	0	0	0		0		0	0	3	3			3		0		0		
		1	1		3		3	2	2	2			2		2		6		
x	x	x	x		x		x	x	x	x			x		x		x		

页故障率=12/20

LRU算法的实现开销很大，必须有硬件的支持，若完全由软件实现其速度至少会降低90%。

4. 最近不用的先淘汰(not used recently, NUR):淘汰最近一段时间未用过的页面。

在实现时, 为每一个页面增加两个硬件位, 分别为

- ①引用位=0, 此页未被访问过;
- ②引用位=1, 此页曾被访问过;
- ③修改位=0, 此页未被修改过;
- ④修改位=1, 此页曾被修改过。

每隔固定时间将所有引用位清零。

当要淘汰某一页面时，按照下面的次序进行选择：

- ①引用位=0，修改位=0；直接淘汰
- ②引用位=0，修改位=1；淘汰之前写回外存
- ③引用位=1，修改位=0；直接淘汰
- ④引用位=1，修改位=1；淘汰之前写回外存

NUR和LRU算法比较:

Reference string: 2, 3, 5, 6, 4, 2, 5, 6, 7, 5, 6, 8,

↑
引用清0

↑
选择淘汰

为进程分配6个物理页框

LRU: 3

NUR: 2, 3, 4任意

5. 最不经常使用的先淘汰(LFU)：淘汰使用次数最少的页面

依据：活跃访问页面应有较大的访问次数

实现：计数器，调入清0，访问增1，淘汰选取最小者

Suffer：(1) 前期使用多，但以后不用，难换出

(2) 刚调入的页面，引用少，被换出可能大.

6. 最频繁使用算法 (MFU)：与最不经常使用算法LFU相反，淘汰使用次数最多的页面。

依据：计数器值最小的页面很可能刚刚调入内存，正待被使用。而使用多的页面可能已经用完了，因而淘汰时取计数器值最大的页面。

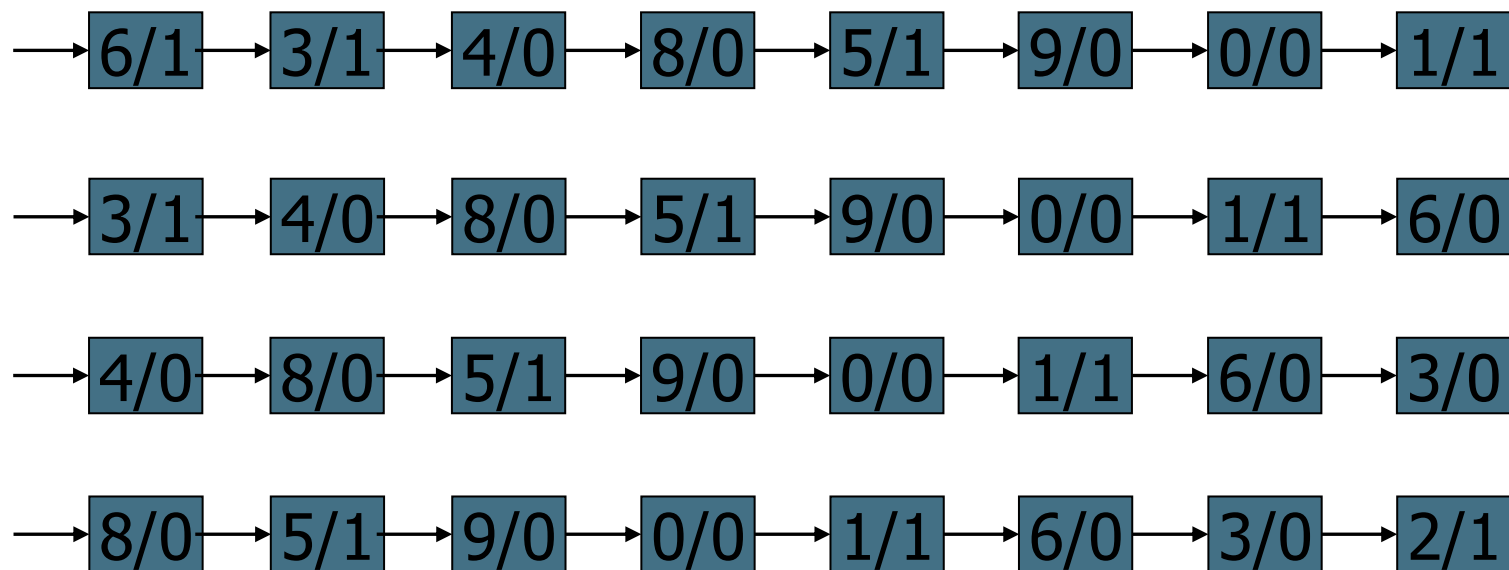
Suffer：程序有些成分是在整个程序运行中都使用的。

LFU和MFU实现开销很大。

7. 二次机会(second chance)

- 淘汰装入最久且最近未被访问的页面
- 实现时：采用拉链数据结构。

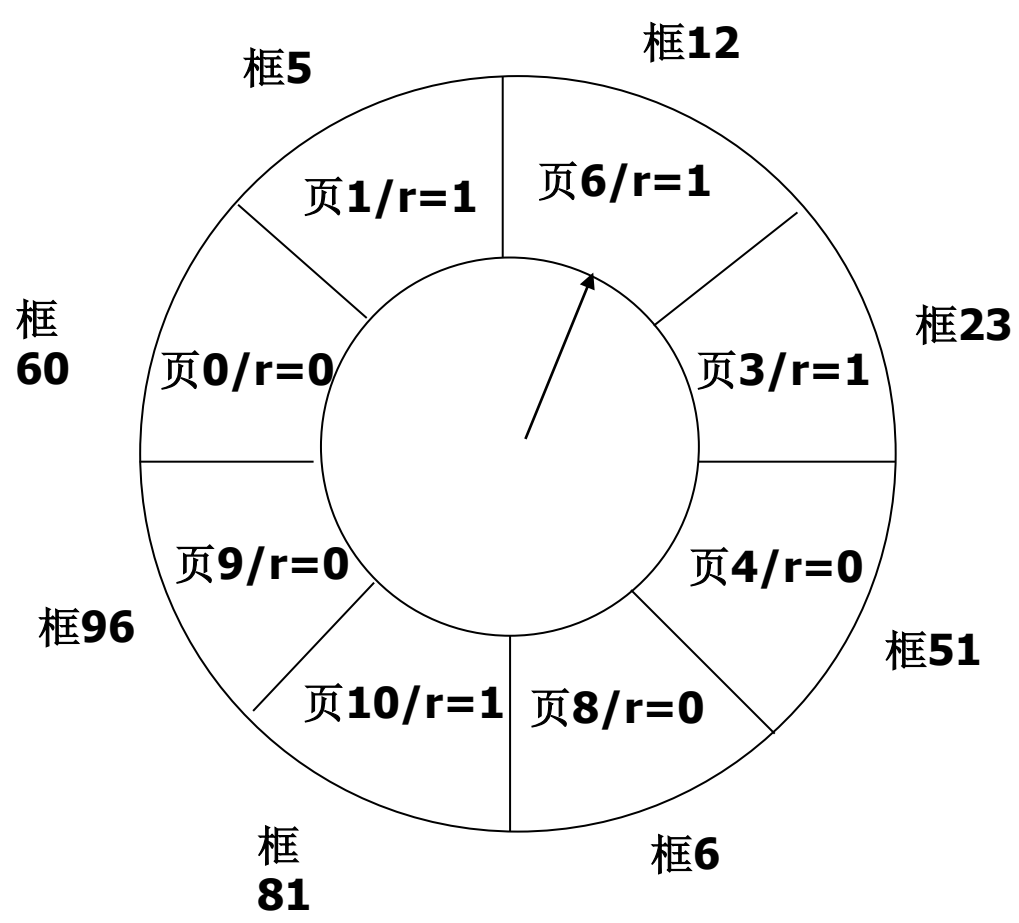
当访问第**2**个页面时



8. 时钟算法(clock algorithm)

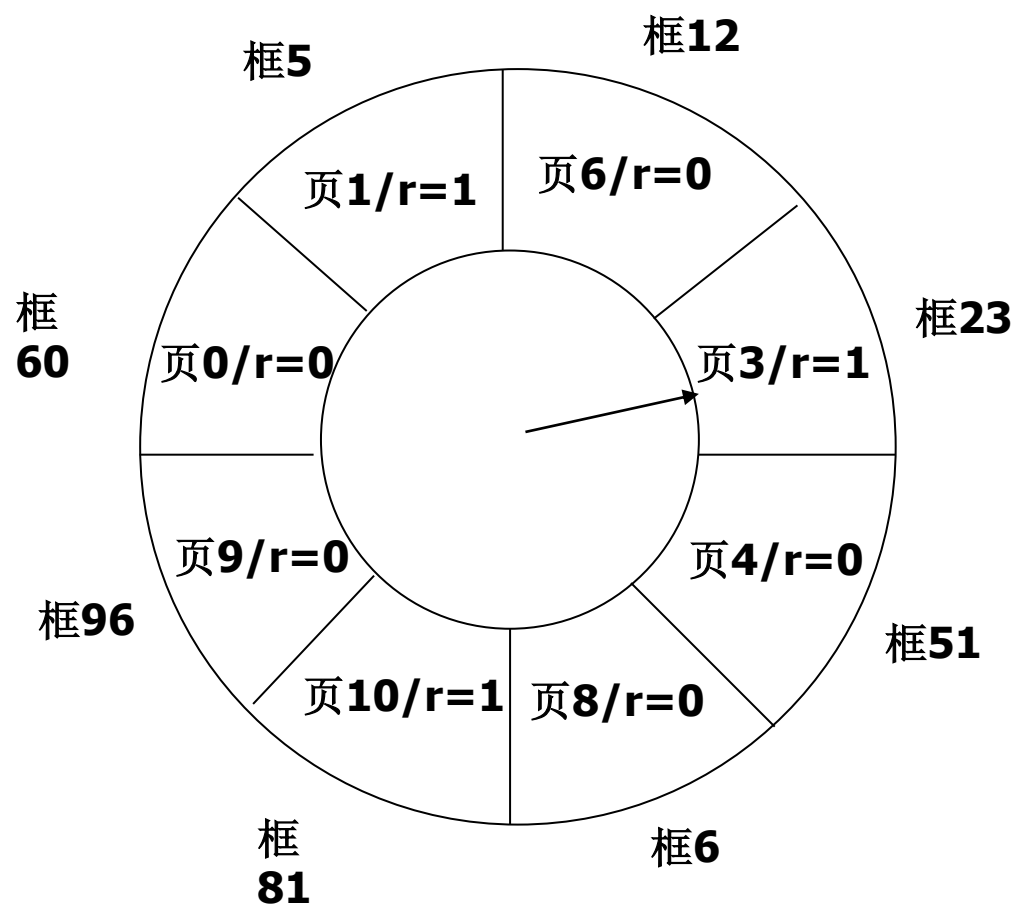
- 将页面组织成环形，有一个指针指向当前位置。
 - 每次需要淘汰页面时，从指针所指的页面开始检查
 - 如果当前页面的访问位为0，即从上次检测到目前，该页没有访问过，则将该页替换
 - 如果当前页面的访问位为1，则将其清0，并顺时针移动指针到下一个位置
 - 重复上述步骤直至找到一个访问位为0的页面

时钟算法



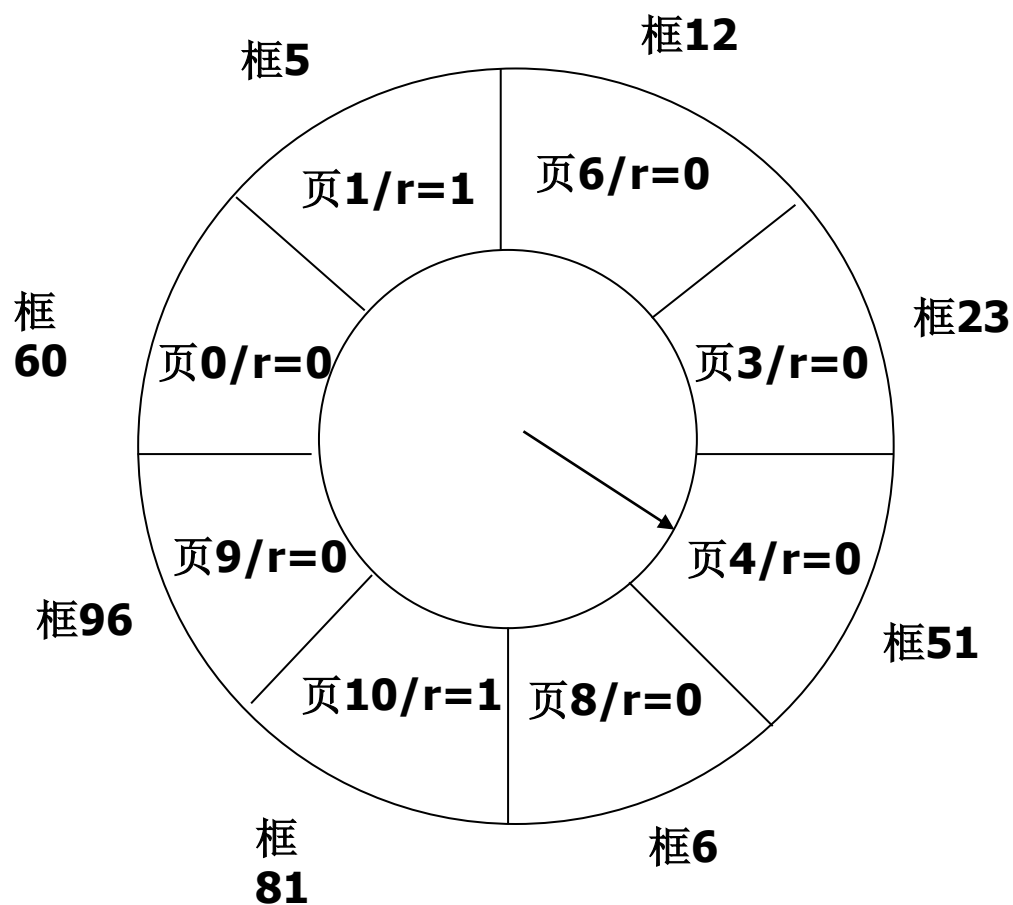
访问第**18**页

时钟算法



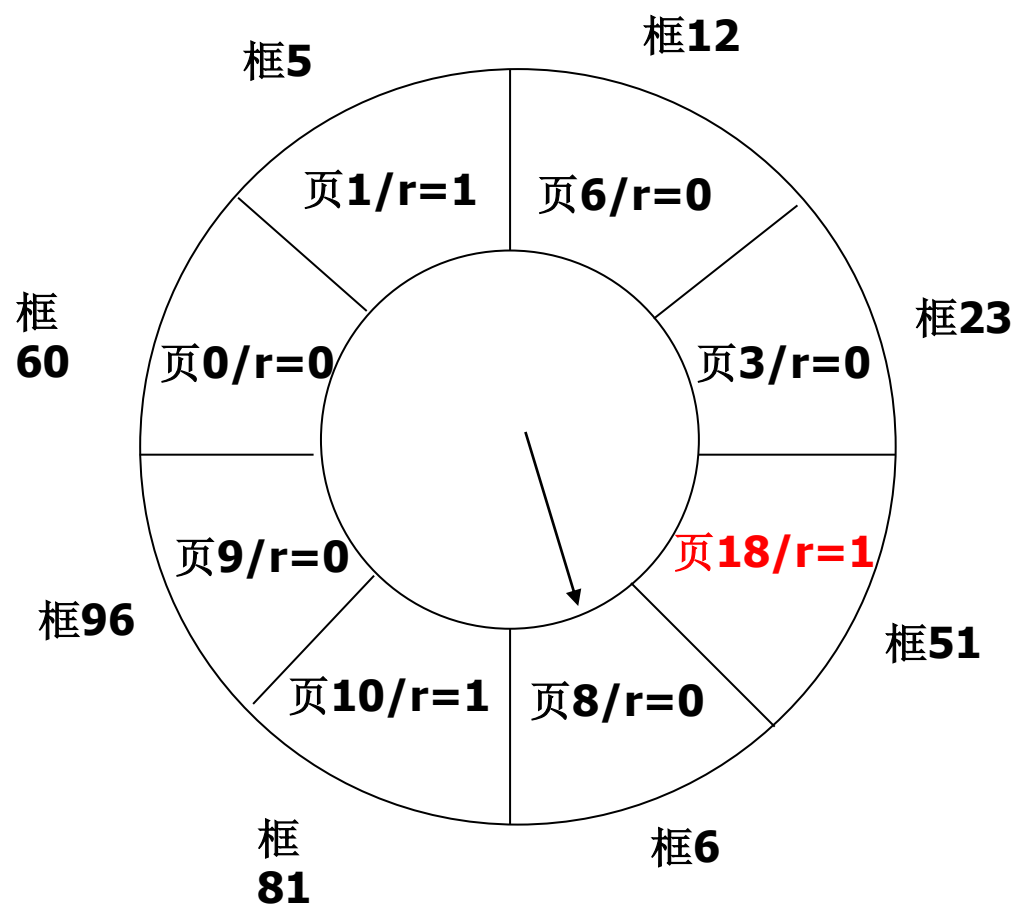
访问第**18**页

时钟算法



访问第**18**页

时钟算法



替换第4页

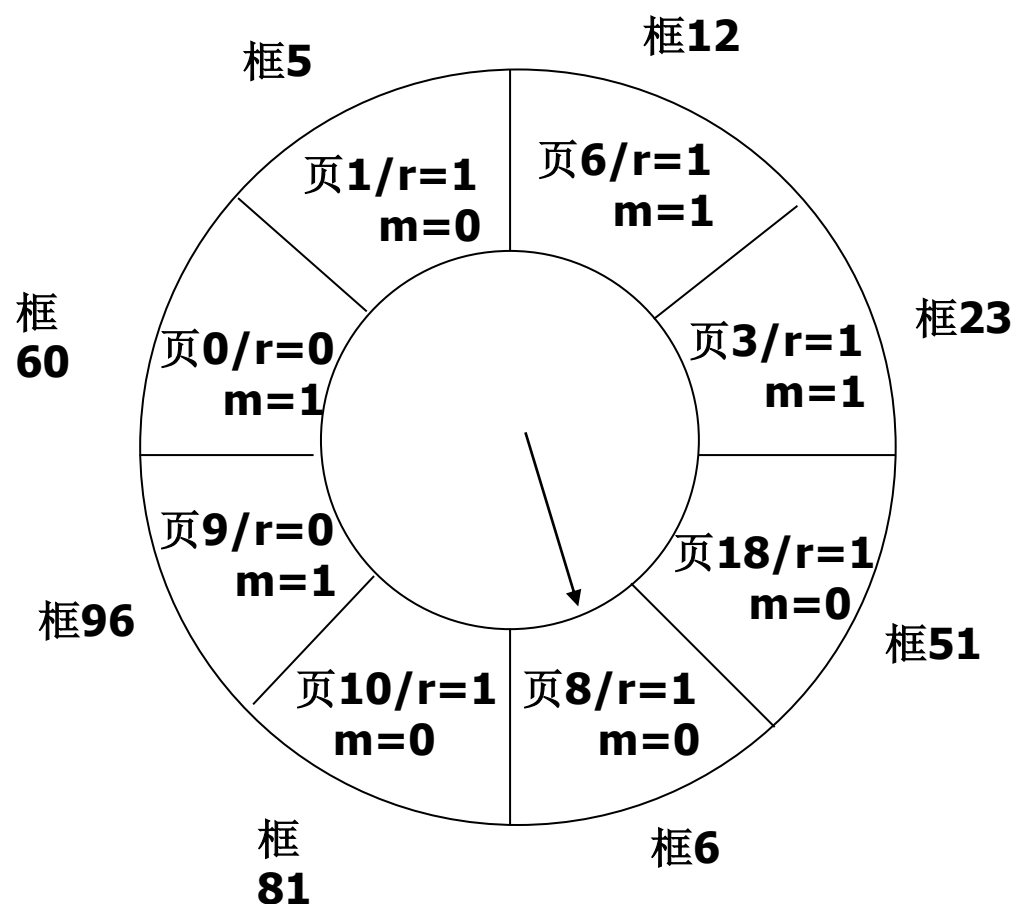
改进的时钟算法

- 考虑修改标志 m
 - $r=0, m=0$: 最佳
 - $r=0, m=1$: 次佳, 淘汰前回写
 - $r=1, m=0$: 再次
 - $r=1, m=1$: 最后, 淘汰前写回

改进的时钟算法

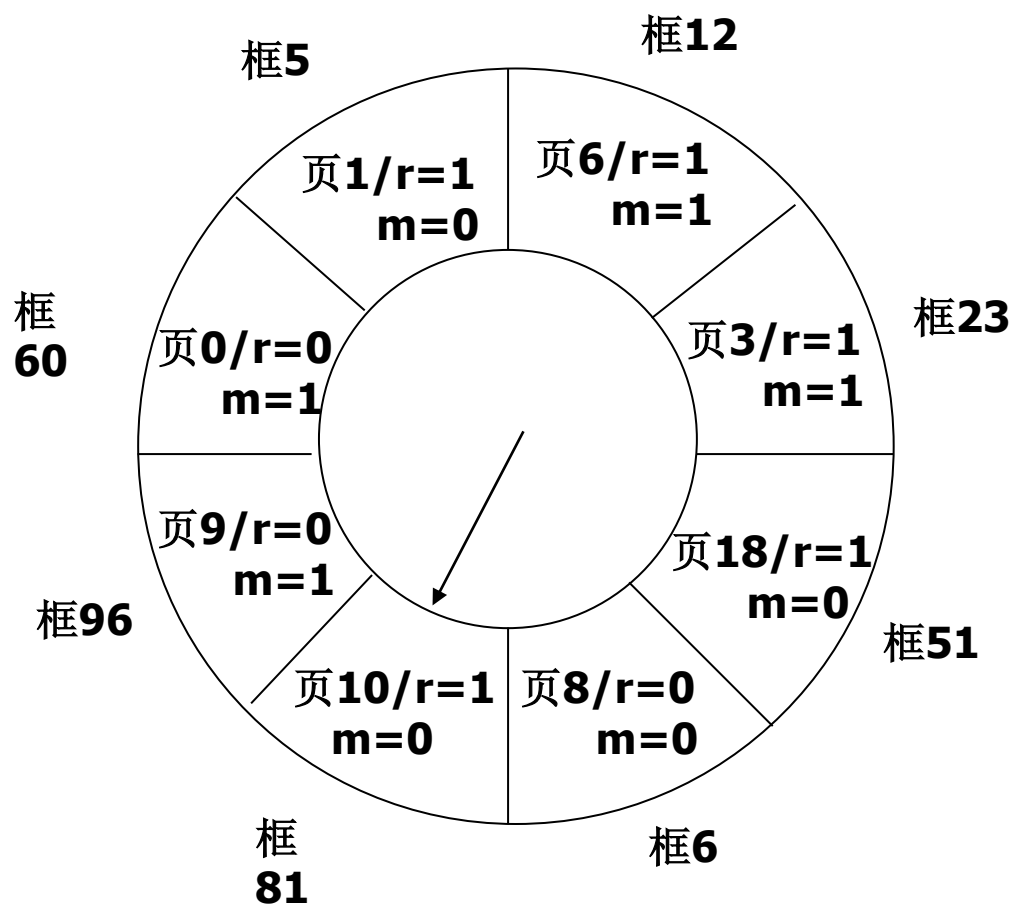
- 步骤1
 - 由指针当前位置开始扫描，选择最佳淘汰页面，不改变引用位，将第一个遇到的 $r=0$ 且 $m=0$ 的页面作为淘汰页面
- 步骤2
 - 如步骤1失败，再次从原位置开始，找 $r=0$ 且 $m=1$ 的页面，将第一个满足上述要求的页面作为淘汰页面，同时将扫描过页面的 r 位清0
- 步骤3
 - 若步骤2失败，指针再次回到原位置，重新执行步骤1。若还失败再次执行步骤2，此时定能找到

改进的时钟算法



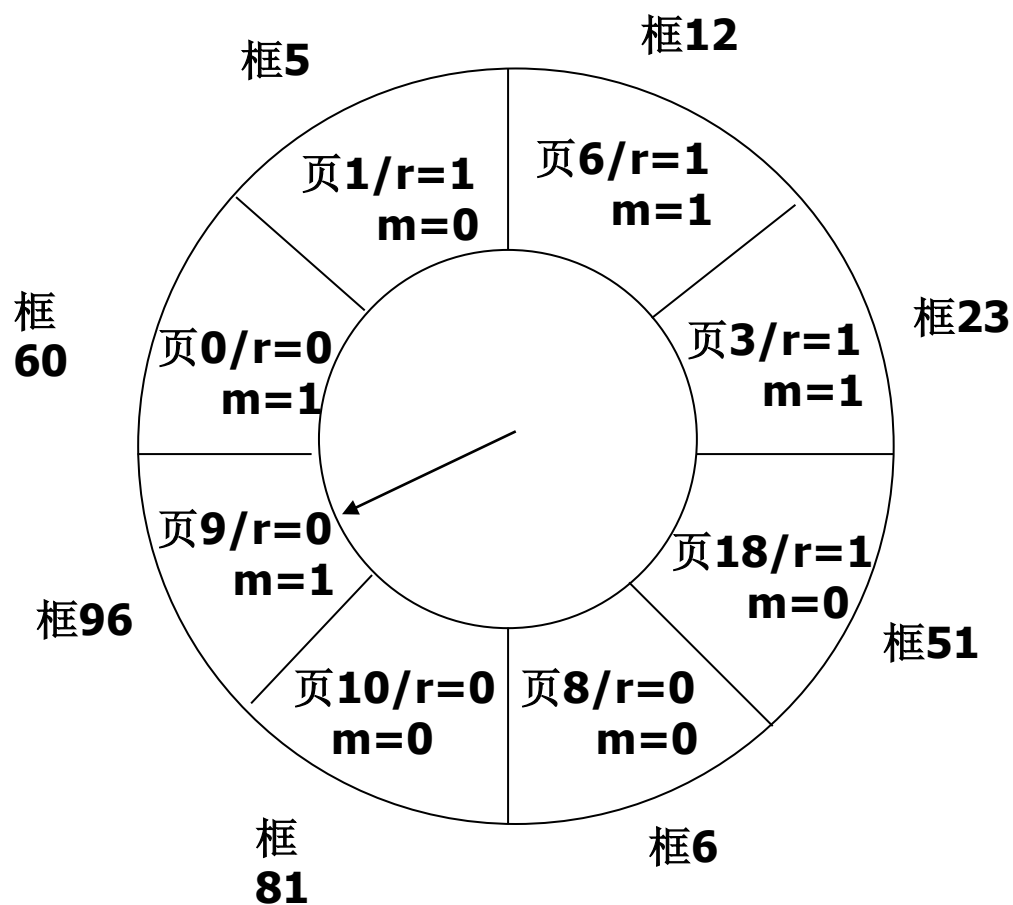
访问第**15**页

改进的时钟算法



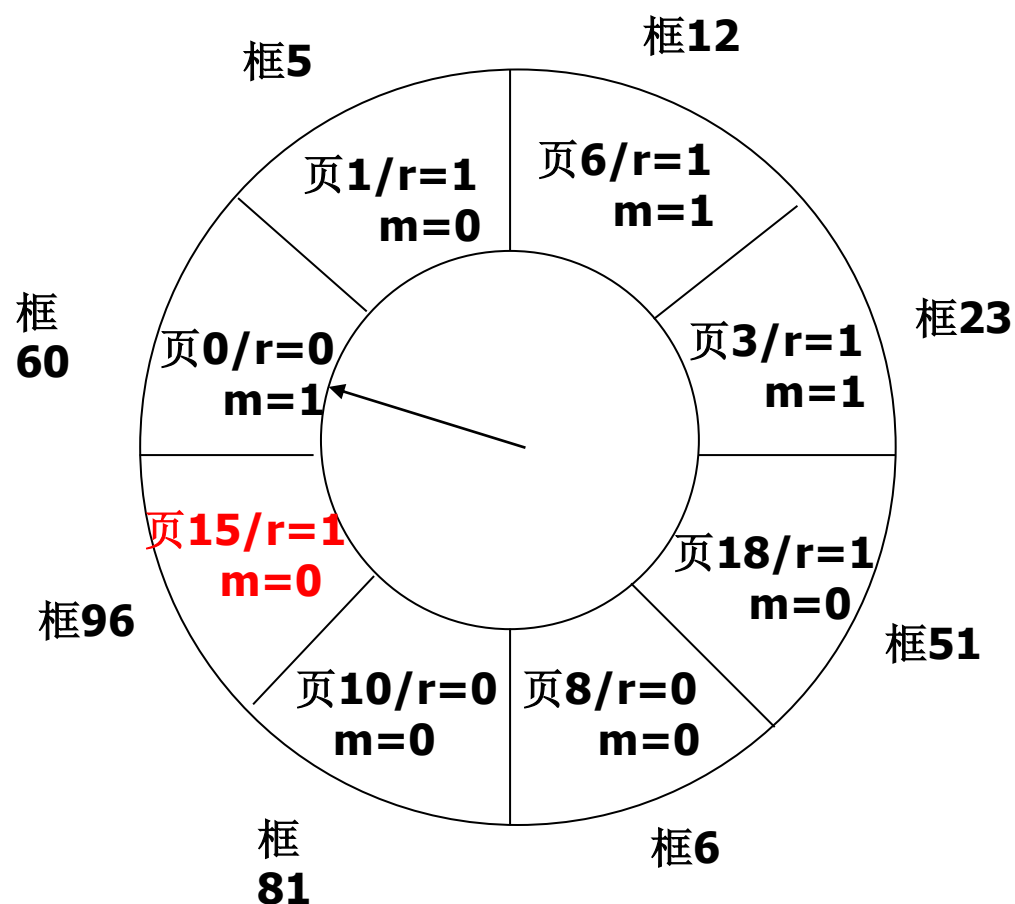
访问第**15**页

改进的时钟算法



访问第**15**页

改进的时钟算法



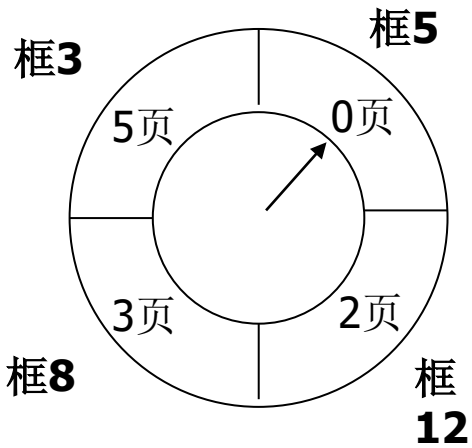
改进的时钟算法

- 时钟算法
 - 尽量淘汰没被访问过的
- 改进的时钟算法
 - 尽量淘汰没被访问过的且没被修改过的

2010年考研试题

- 某虚拟页式系统，进程空间和内存空间都是64k，页长1K，某进程6个页，内存分配4个页框，采用局部置换，280时刻页表和Clock数据结构如下：

逻辑页号	页框号	装入时间	访问标志
0	5	110	1
1	-	-	-
2	12	160	1
3	8	230	1
4	-	-	-
5	3	80	1



(顺时针)

2010年考研试题

- (1) 280时刻访问13B7H, 逻辑页号是多少?
- (2) 采用FIFO置换算法, 物理页框号是多少? 物理地址是多少?
- (3) 采用CLOCK置换算法, 页框号是多少? 物理地址是多少?

2010年考研试题

- (1) 逻辑地址13B7H化为二进制数为0001001110110111，其中后10位为页内地址，前6位为逻辑页号，即逻辑页号为4。
- (2) 4号页不在内存，发生缺页中断，按FIFO置换算法，应置换第5页，所得页框号3，形成物理地址0000111110110111，划成16进制为0FB7H。
- (3) 采用CLOCK置换算法，淘汰第0页，得页框5，形成物理地址为0001011110110111，换成16进制为17B7H。

7.2.1.6 颠簸(thrashing)

- 页面在内存与外存之间频繁换入换出。

原因：(1) 分给进程物理页框过少；

(2) 淘汰算法不合理；

(3) 程序结构不好。

处理：(1) 增加分给进程物理页框数；

(2) 改进淘汰算法；

(3) 改善程序结构。

关于程序结构对页故障的影响

```
int a[1024][1024];
```

清零操作:

```
(1) for(i=0;i<1024;i++)
    for(j=0;j<1024;j++)
        a[i][j]=0;
```

```
(2) for(i=0;i<1024;i++)
    for(j=0;j<1024;j++)
        a[j][i]=0;
```

```
a[0][0];
a[0][1];
...
a[0][1023];
a[1][0];
a[1][1];
...
a[1][1023];
a[2][0];
a[2][1];
...
a[2][1023];
...
a[1023][0];
a[1023][1];
...
a[1023][1023];
```

1页

1页

1页

1页

(假定 1 页长 1K)

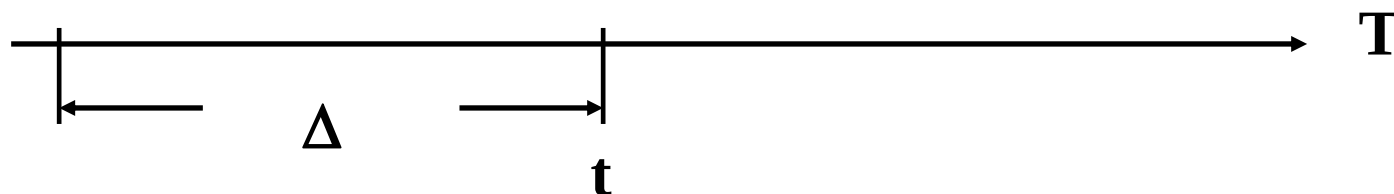
思考题：

在某些虚拟页式存储管理系统中，内存中总保持一个空闲的物理页框，这样做有什么好处？

7.2.1.7 工作集模型(working set model)

工作集(working set): 进程在一段时间内所访问页面的集合。

...2 6 1 5 7 7 7 7 5 1 6 2 2 1 2 3 ... (page reference)



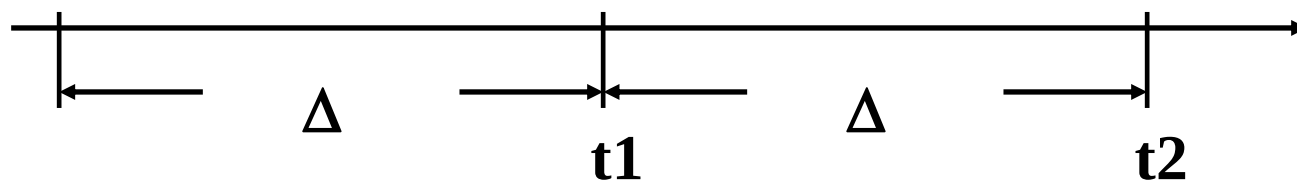
$$WS(t, \Delta) = \{5, 7, 1, 6, 2\}$$

Δ : 称为窗口尺寸(window size)。

为使程序有效运行, 工作集应能放入内存。

工作集与时间有关：

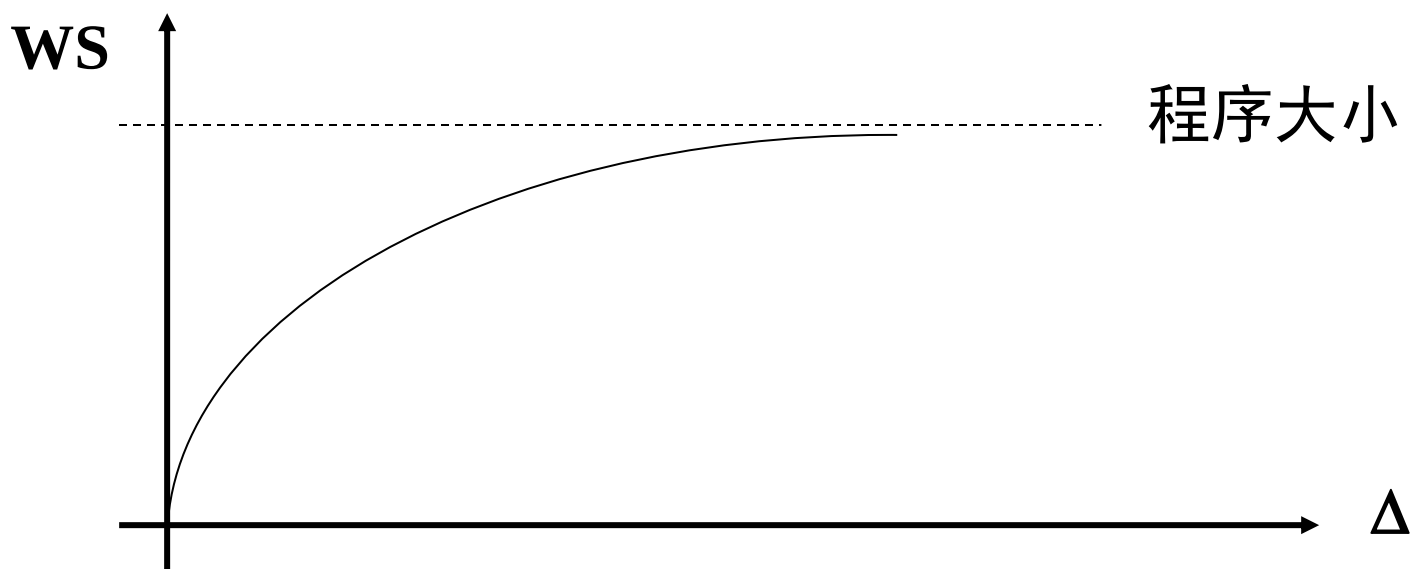
...2 6 1 5 7 7 7 7 5 1 6 2 2 1 2 3 4 4 4 3 4 3 4 4 4 1 3 4 ...



$$WS(t_1, \Delta) = \{5, 7, 1, 6, 2\}$$

$$WS(t_2, \Delta) = \{2, 3, 4\}$$

工作集与窗口尺寸有关：



Δ 过小：活动页面不能全部装入，页故障率高；

Δ 过大：内存浪费。

Madnick, Donovan建议: $\Delta=1$ 万次访内。

实现: 页表中增加访问位:

页表:

...	访问位

Δ 开始时, 全部清0,

访问: 置1,

Δ 结束时(新 Δ 开始时)访问标志为1的, 为该 Δ 期间工作集,

工作集估计：

令 w_n 表示第 n 个 Δ 周期的实际工作集大小， τ_n 为第 n 个 Δ 周期的估计工作集大小， α 为0-1之间的数，则第 $n+1$ 个 Δ 周期的估计工作集大小 τ_{n+1} 定义如下：

$$\tau_{n+1} = \alpha w_n + (1 - \alpha) \tau_n$$

工作集模型能够很好地指导页面的分配，从而防止颠簸。
但实现开销较大。

7.2.1.8 页故障率反馈模型

PFFB (Page Fault Feed Back)：动态调整页面的分配

页故障率高(达到某上界阈值)：内存页框少，增加

页故障率低(达到某下界阈值)：内存页框多，减少

7.2.1.11 非均匀存储器访问

■问题：

- 单处理机系统：内存单元是平等的
- 多处理机系统：每个CPU有自己的局部存储器，访问局部存储器的速度明显比访问非局部内存快——非均匀存储器访问(NUMA)

使用：在NUMA体系结构中，CPU为进程分配页框应尽量优先使用该CPU的局部存储器，当CPU空闲时，尽量优先考虑上次在本CPU上运行的进程，原因是这样的进程所拥有的内存更可能是本CPU的局部存储器。

页面大小的选择

- 页面太大
 - 浪费内存，极限是分区存储
- 页面太小
 - 页面增多，页表长度增加，浪费内存
 - 换页频繁，系统效率低
- 页面的常见大小
 - 2的整数次幂：1KB，2KB，4KB.

影响缺页次数的因素

- 淘汰算法
- 分配给进程的页框数
 - 页框数小，越容易缺页
- 页本身的大小
 - 页面越小，容易缺页
- 程序的编制方法
 - 局部性越好，越不容易缺页
 - 跳转或分支越多越容易缺页

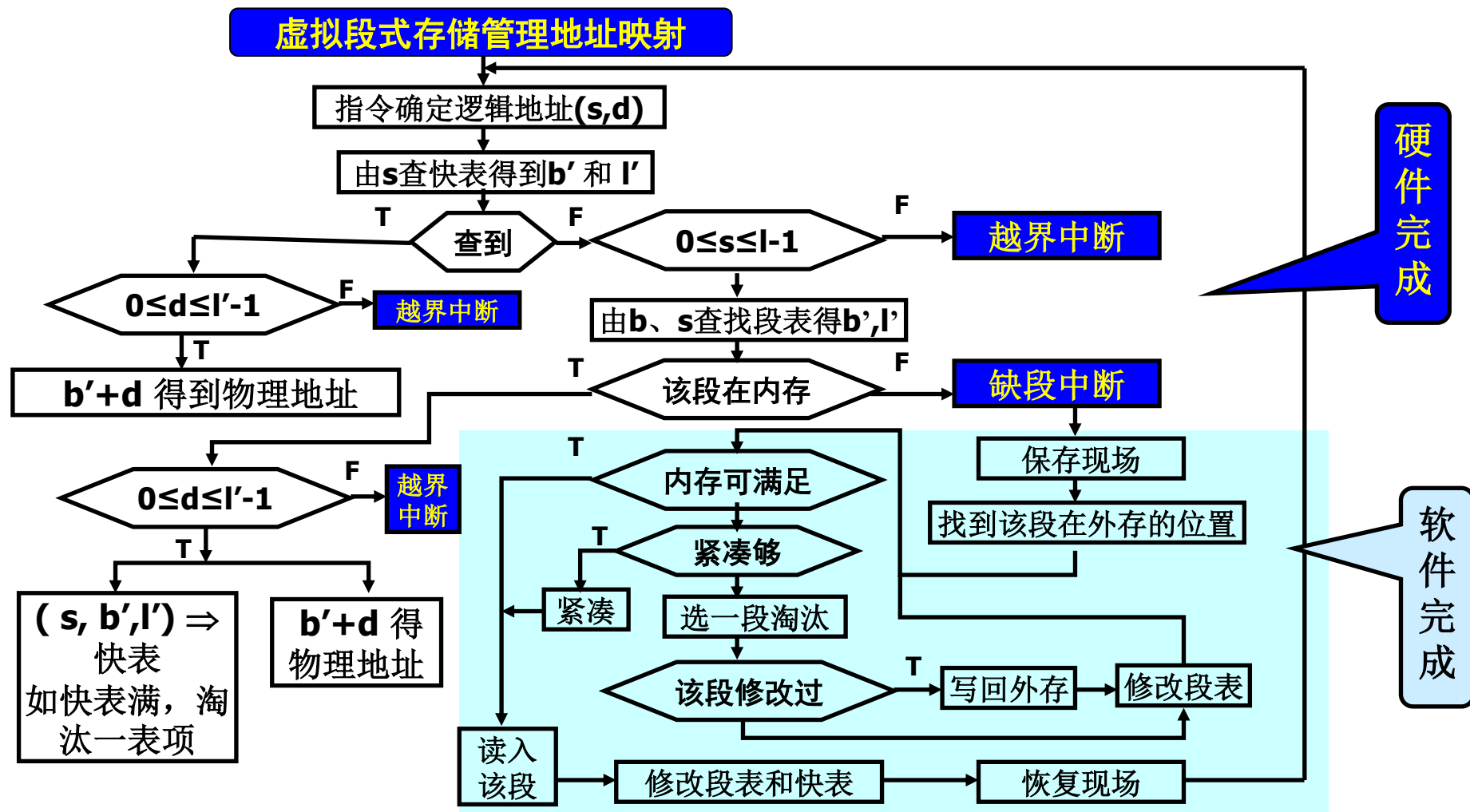
页式系统的不足

- 页面划分无逻辑意义
- 页的共享不灵活
- 页内碎片

7.2.2 虚拟段式存储系统

- 进程运行前，全部装入外存，部分装入内存，访问段不在内存时，发生缺段中断
- 若内存空间不够
 - 紧凑：将内存中的所有空闲区合并
 - 淘汰：将内存中的某段移至外存。

7.2.2 虚拟段式存储系统(Cont.)



(1)段表的改进:

段号	段长	内存首址	外存首址	权限	内外标识	修改标志
...	...	...	...	...	...	...
s	l'	b'	b''	{rwe}	(0,1)	(0,1)
...	...	...	...	...	...	...

(2)快表的改进:

段号	段长	内存首址	访问权限	修改标志
...	...	...	...	...
s	l'	b'	{rwe}	(0,1)
...	...	...	...	...

7.2.2.2 段的动态连接(dynamic linking)

- 当遇到访问外段的指令时，需要进行连接
- 动态连接 vs. 静态连接
 - 静态连接：运行前连接，由link完成；一个程序共有多少个段是确定的，因而link可以为每一个段分配一个段号；
 - 动态连接：运行时连接，由OS完成。（在程序运行过程中需要某一个段时才将该段连接上）；一个程序共有多少个段是不确定的，因而段名到段号的转换需要由操作系统来完成的。
- 静态连接的缺点
 - 连接时间长
 - 目标代码长
 - 连接段可能并不执行(未用到)

动态连接的实现(Multics为例)

段名-段号对照表：每进程一个
记录当前已连接段的表目

段名	段号
...	...
sname	snum
...	...

符号表：每段一个

符号名	段内位移
...	...
func	150
...	...

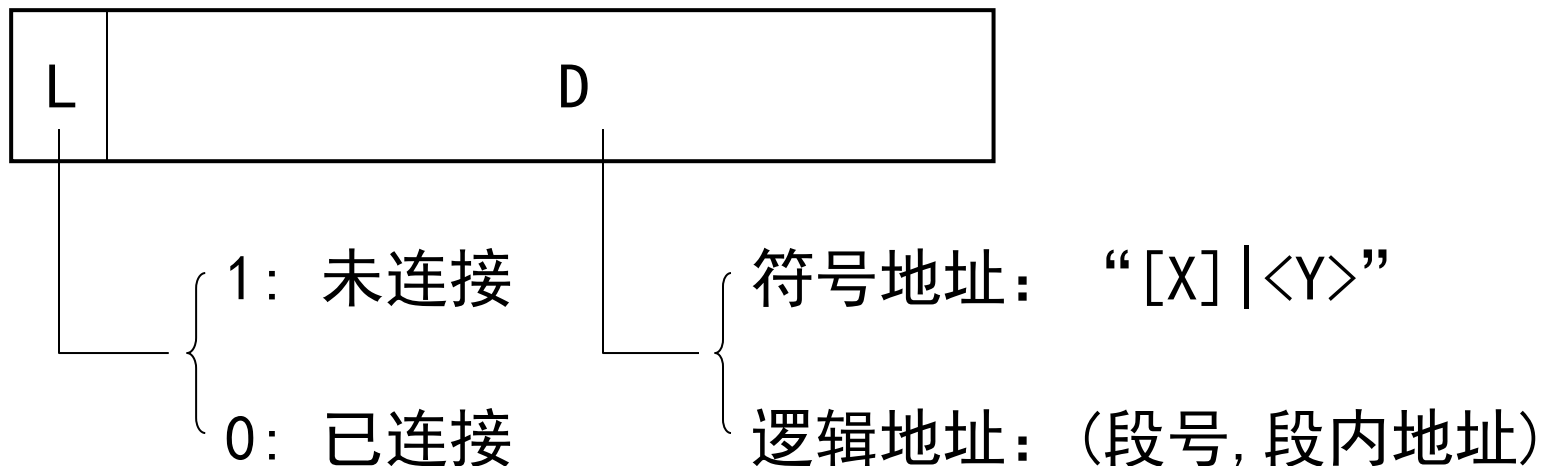
段形式：

符号表
目标代码 或者数据



(1) 编译(汇编)时:

遇到访问外段指令，采用间接寻址，指向一个间接字：



(2) 执行时:

遇到间接指令，且L=1，发生连接中断，处理程序：

- (a) 由D取出符号地址；
- (b) 由段名查段名-段号对照表，是否分配段号。

♣ 已分配段号：该段以前连接过，从段名段号对照表中取出该段号，然后由段号查段表找到该段，继而由单元名查找该段的符号表，从中取出段内地址；

$$(\text{段号}, \text{段内地址}) \Rightarrow D, 0 \Rightarrow L$$

♣ 未分配段号：该段以前未连接过，找到该段所在文件，读入内存，分配段号，填写段名-段号对照表，填写段表，转(b)

动态连接后，重新执行指令，发现连接字 $L = 0$ ，为一般的间接地址。

例子：汇编前：

[W]段

...

Load 1, [X]|<Y>

...

Load 2, [X]|<Z>

...

[X]段

Y: 1234

...

Z: 5678

...

假设[W]段已经连接并在内存

汇编后，连接前：

段表

段号	...	段首址
0		
1		
2		
3		
...	...	
...		

段名:段号对照表

段名	段号
[Main]	0
[A]	1
[W]	2

[W]段（段号=2）

Load *1, 2|100;
...
Load *2, 2|150;
...

100:

1	...	2	200
---	-----	---	-----

150:

1	...	2	250
---	-----	---	-----

...
200: “7[X]|<Y>”
...
250: “7[X]|<Z>”

[X]段

<Y>	300
<Z>	400
300	1234
400	5678

第一次连接后：

段表

段号	...	段首址
0		
1		
2		
3		
...	...	

段名:段号对照表

段名	段号
[Main]	0
[A]	1
[W]	2
[X]	3

[W]段 (段号=2)

Load *1, 2|100;
...
Load *2, 2|150;
...

100:

1	...	2	200
---	-----	---	-----

150:

1	...	2	250
---	-----	---	-----

...
200: “7[X]|<Y>”
...
250: “7[X]|<Z>”

[X]段

<Y>	300
<Z>	400
300	1234
400	5678

第一次连接后：

段表

段号	...	段首址
0		
1		
2		
3		
...	...	

段名:段号对照表

段名	段号
[Main]	0
[A]	1
[W]	2
[X]	3

[W]段 (段号=2)

Load *1, 2|100;
...
Load *2, 2|150;
...

100:

0	...	3	300
---	-----	---	-----

150:

1	...	2	250
---	-----	---	-----

...
200: “7[X]|<Y>”
...
250: “7[X]|<Z>”

[X]段

<Y>	300
<Z>	400
300	1234
400	5678

第二次连接后：

段表

段号	...	段首址
0		
1		
2		
3		
...	...	
...		

段名:段号对照表

段名	段号
[Main]	0
[A]	1
[W]	2
[X]	3

[W]段 (段号=2)

Load *1, 2|100;
...
Load *2, 2|150;
...

100: 0 ... 3 300

150: 0 ... 3 400

...

200: "7[X]|<Y>"

...
250: "7[X]|<Z>"

[X]段

<Y>	300
<Z>	400
300	1234
400	5678

动态连接问题

- 动态连接与共享的矛盾
 - 动态连接：修改连接字（代码）
 - 段的共享：要求纯代码（pure code）
- 解决方法
 - 共享代码分为“纯段”和“杂段”
 - 纯段共享，
 - 杂段私用。

7.2.3 虚拟段页式存储系统

- 虚拟页式存储管理
 - 简化存储分配，消除外碎片
 - 只提供一维地址，进程空间不能动态扩充，无法实现段的动态连接
 - 虽然能够实现存储共享和保护，但以页为单位，页长度和程序基本单位不等，实现麻烦
- 虚拟段式存储管理
 - 一个段对应一个程序单位，是二维地址
 - 方便存储共享和保护
 - 可以实现段长度的动态变化，以及段间的动态连接
 - 但对存储空间的分配和去配较复杂，可能形成碎片
- 虚拟段页式存储管理
 - 保持优点，克服缺点
 - 过于复杂，实现开销大，需要硬件提供更多支持

7.2.3 虚拟段页式存储系统

- 考虑
 - 段的动态连接
 - 段的共享
 - 段长度的动态变化

所需表目：

(1) 段表：每进程一个，动态变化，每连接一个新段时，增加一个新的表项

段号	页表长度	页表首址	访问权限	扩展标志	共享段入口
	...	...	...	...	...

(2) 页表：每段一个，进程开始时只为主程序段建立页表，其他段的页表在段连接时建立

逻辑页号

页框号	外存块号	内外标志	修改标志
...	...	...	...

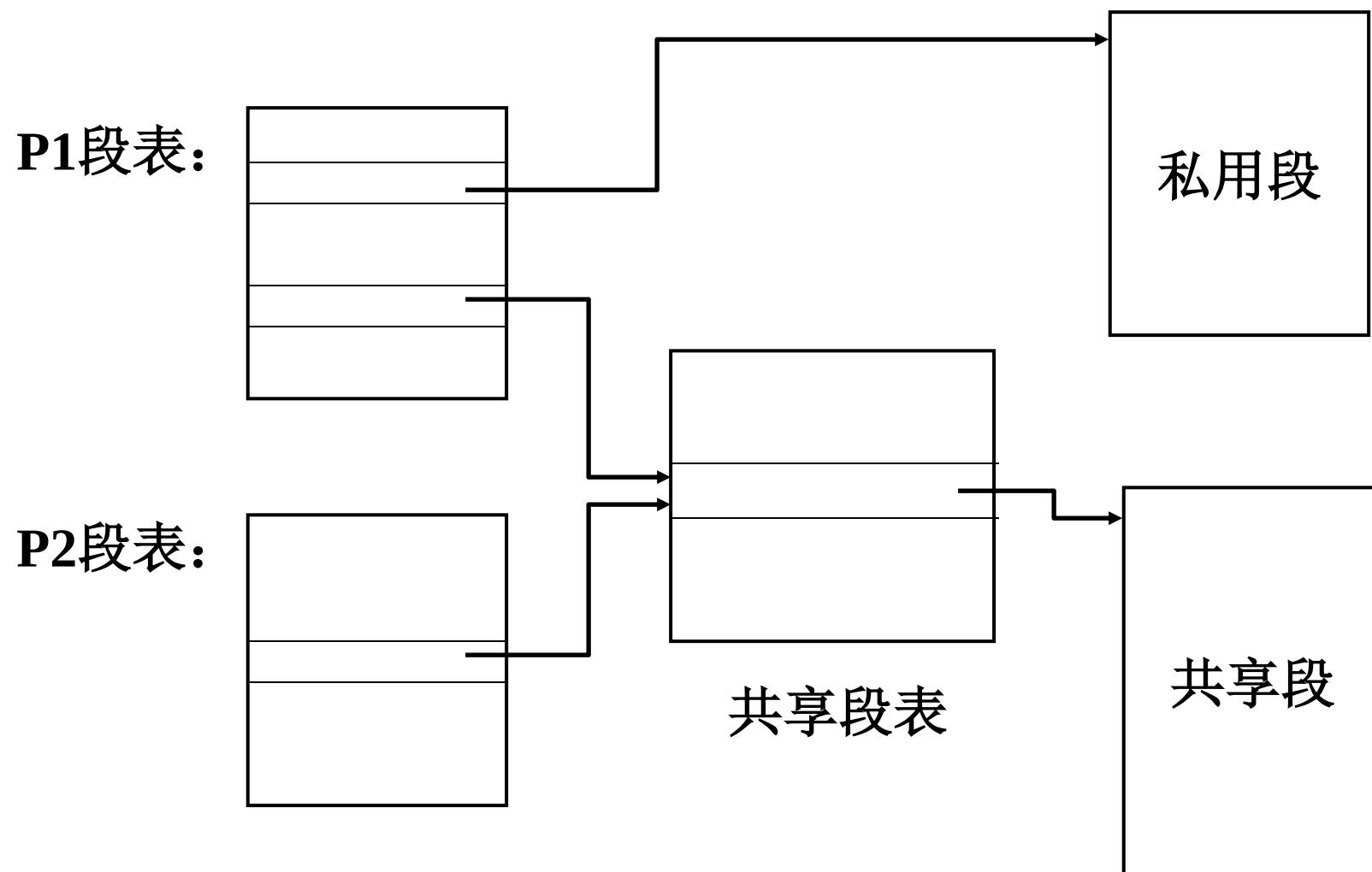
(3) 总页表：即位示图，内存和外存各一个

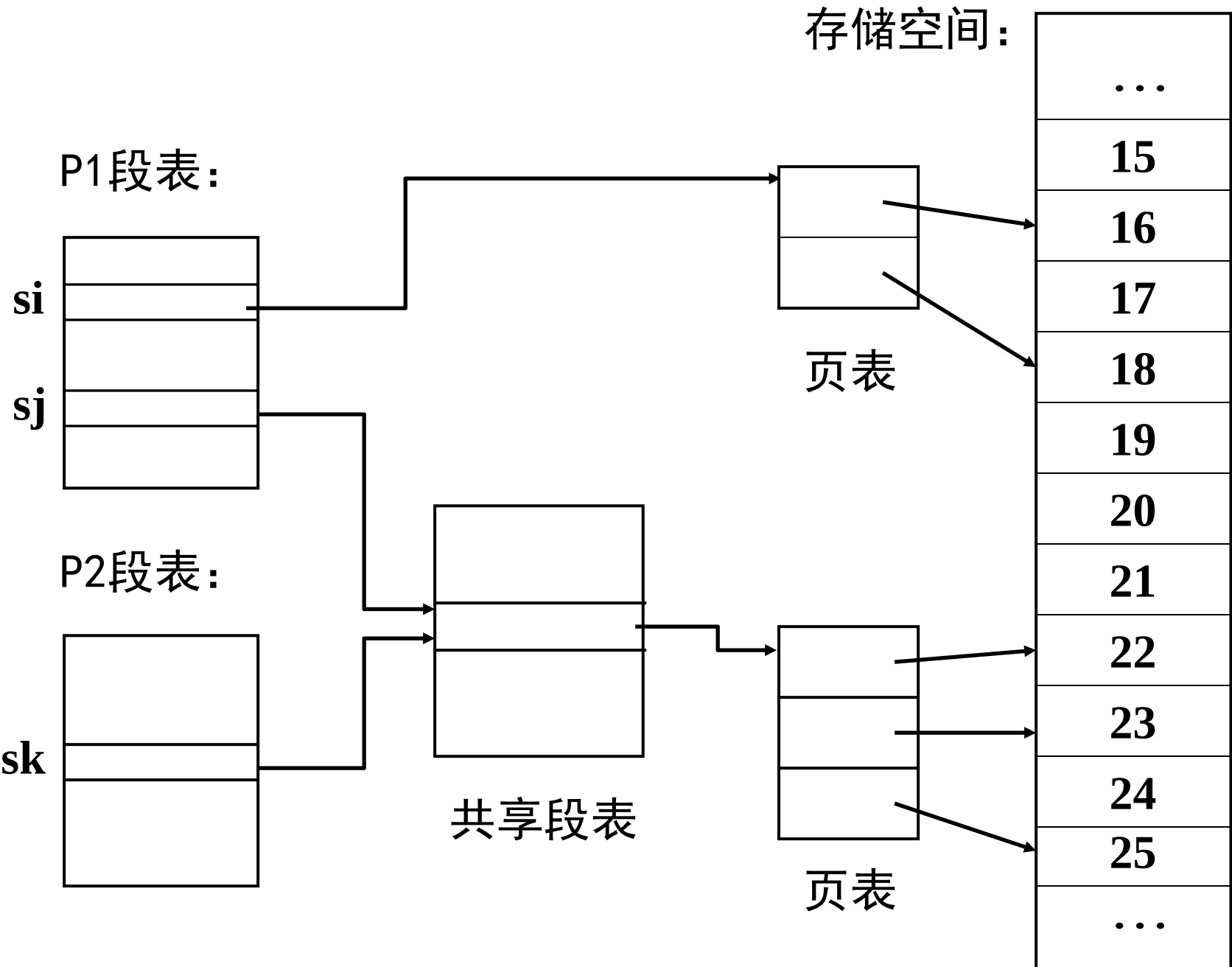
(4) 共享段表：系统一个，记录所有的共享段

段名	页表长度	页表首址	扩充标志	共享计数
	...	...	...	...

(5) 段名-段号对照表：每进程一个

段表和段的对应关系：





所需寄存器：

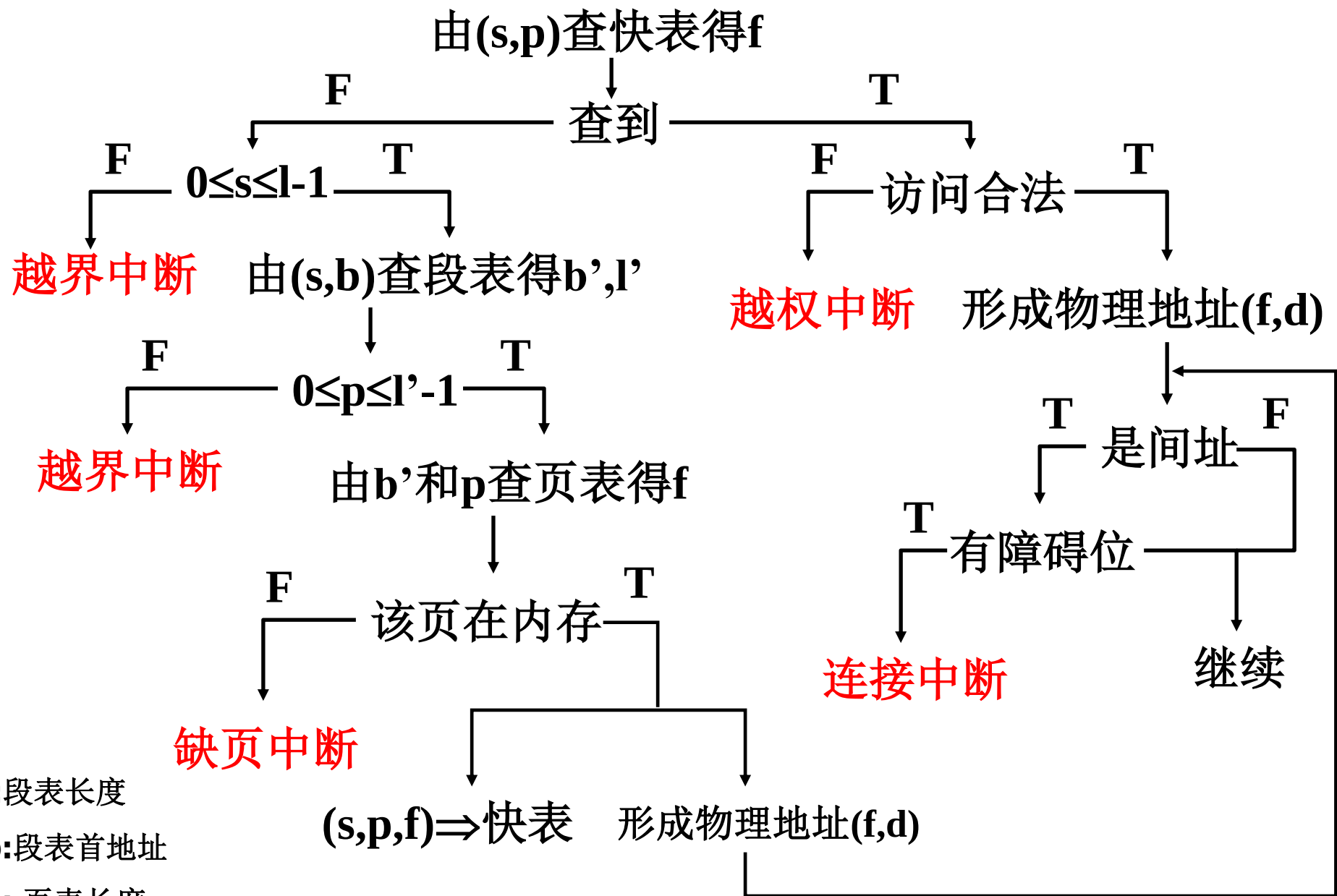
- (1) 段表长度寄存器：保存正运行进程段表长度
- (2) 段表首址寄存器：保存正运行进程段表首址
- (3) 快表

段号	逻辑页号	页框号	访问权限	修改标志
...	...	...	...	...

地址映射：

$$\sigma: (s, p, d) \rightarrow (f, d) \cup \{\Omega\}$$

逻辑地址 $(s, p, d) \Rightarrow$ 物理地址 (f, d)



l:段表长度
b:段表首地址
l': 页表长度
b': 页表首地址

段格式：

共享标志
权限说明
扩展标志

} 段头

符号名表

段内容

中断处理

1. 连接中断

(1) 所有进程均未连接(共享段表、段名-段号对照表均无)，查文件目录找到该段，为该段建立页表，由文件读入外存swap，部分页读入内存，填写页表；为该段分配段号 s ，填段名-段号对照表，如是共享段填共享段表，填段表，由符号名查符号表得 (p, d) ，形成逻辑地址。

(2) 其它进程连接过，本进程未连接过(共享段表有，段名-段号对照表无)，分配段号 s ，填段名-段号对照表，填段表(访问权限，指向共享段表)，共享计数加1，由符号名查符号表得 (p, d) ，形成逻辑地址。

(3) 本进程连接过(段名-段号对照表有，共享段表有或无)，由符号名查符号表得 (p, d) ，形成逻辑地址。

2. 缺页中断

分配页框，调入所需页面，更新页表和总页表。

3. 越界中断

(1) 段号越界：访问不存在的段，地址错误，程序被中止。

(2) 页号越界：访问段内不存在的页，根据段扩充标志判断该段是否可以扩充。如可扩展，扩展该段(增加页)，修改页表和段表(或共享段表)；如不可扩展，地址错误，程序被中止。

4. 越权中断

违反段的访问权限，程序将被中止。

特性 管理方式	地址 空间	存储 分配	存储 碎片	存储 共享	存储 保护	动态 扩充	动态 连接
虚拟页式	一维	简单	无	不便	不便	不可	不可
虚拟段式	二维	复杂	有	方便	方便	可以	可以
虚拟段页式	二维	简单	无	方便	方便	可以	可以

分页对用户是透明的，分段对用户是可见的

7. 系统举例- Linux伙伴堆存储分配算法

- Linux 采用DMA方式进行输入输出操作，DMA不带有地址变换机构，即是在没有地址映射的条件下进行的，因此进程在内存中必须占有连续的页面
- 针对Linux系统，采用伙伴堆存储分配方法。即对内存空闲页面管理时，按照尺寸将连续的页面放在一组，这就是伙伴堆的思想
- 伙伴堆算法用于管理内存中的空闲页框，它是针对内存**碎片问题**而提出的一种稳定高效的分配策略

1. 伙伴堆存储分配算法

(1) Physical memory management

- 页框：将内存静态划分为若干等长的区域, 每个区域为一个页框, 例如: 4KB;
- 块组：Linux将所有空闲页面分为10个块组, 块组编号为 i ($i=0, 1\cdots 9$),
块组 i 中记载长度为 2^i 个页面的连续区域, 即第0组中块的大小为 2^0 (1页), 第1组中块的大小均为 2^1 (2页), 第9组中块的大小均为 2^9 (512页), 同组中的所有块以链表形式存储。
- 空闲区表: `free_area[i]`表示页框数为 2^i 的块组, 其结构为:

空闲块组指针	块组位图指针
--------	--------

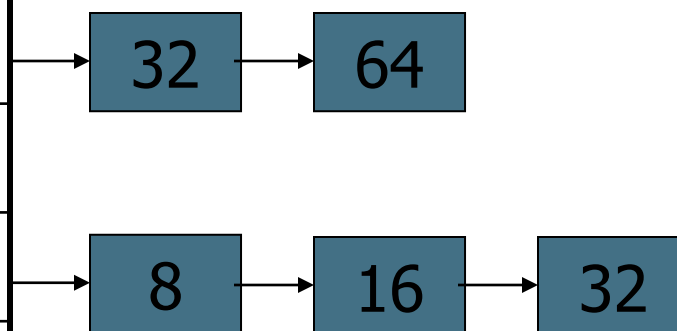
每个空闲块组指针指向该块组内的空闲块组链 (每个块组大小为 2^i 页面)
每个块组位图指针指向该块组内页面的使用情况
- 分配: Buddy heap algorithm
以 2^i 个页框为分配单位 ($2^{i-1} < fn \leq 2^i$),
 fn 为要申请的页框数;

Buddy Heap Implementation

9(2^9)
8(2^8)
7(2^7)
...
3(2^3)
2(2^2)
1(2^1)
0(2^0)

数据结构:

组号 (空闲页数) : 链头指针



相同长度的空闲块
构成一组

■ 块组位图：

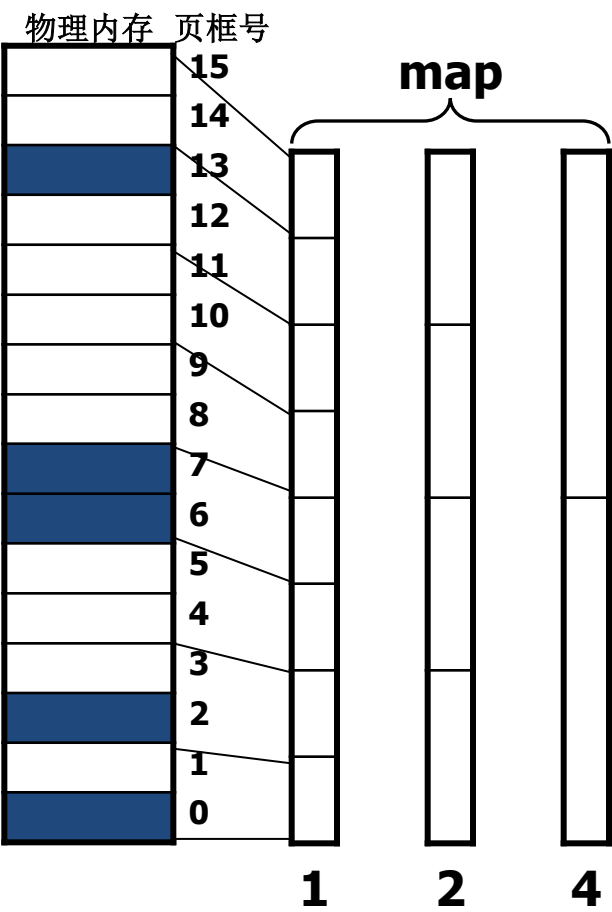
- 对于块组 i ，将内存中的所有页面（包括占用和空闲），按前后顺序两两结合成一对伙伴（Buddy）。即按前后顺序以 2^i 个页面作为一个块组，与其相邻的 2^i 个页面作为另一个块组，那么这两个块组合为一对Buddy。
如：对于块组1，那么0、1页框和2、3页框是一对Buddy；

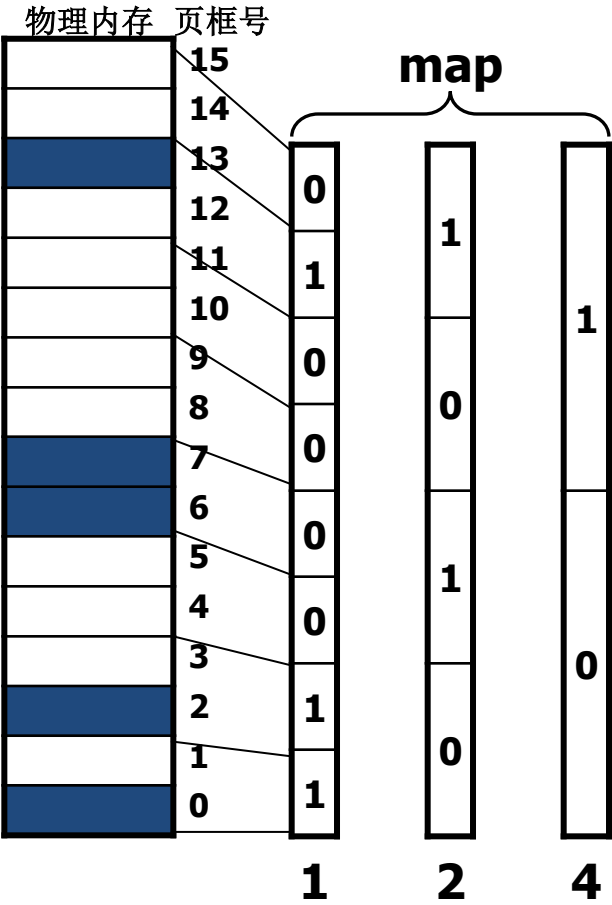
对于块组2，那么0, 1, 2, 3页框与4, 5, 6, 7页框是一对Buddy。

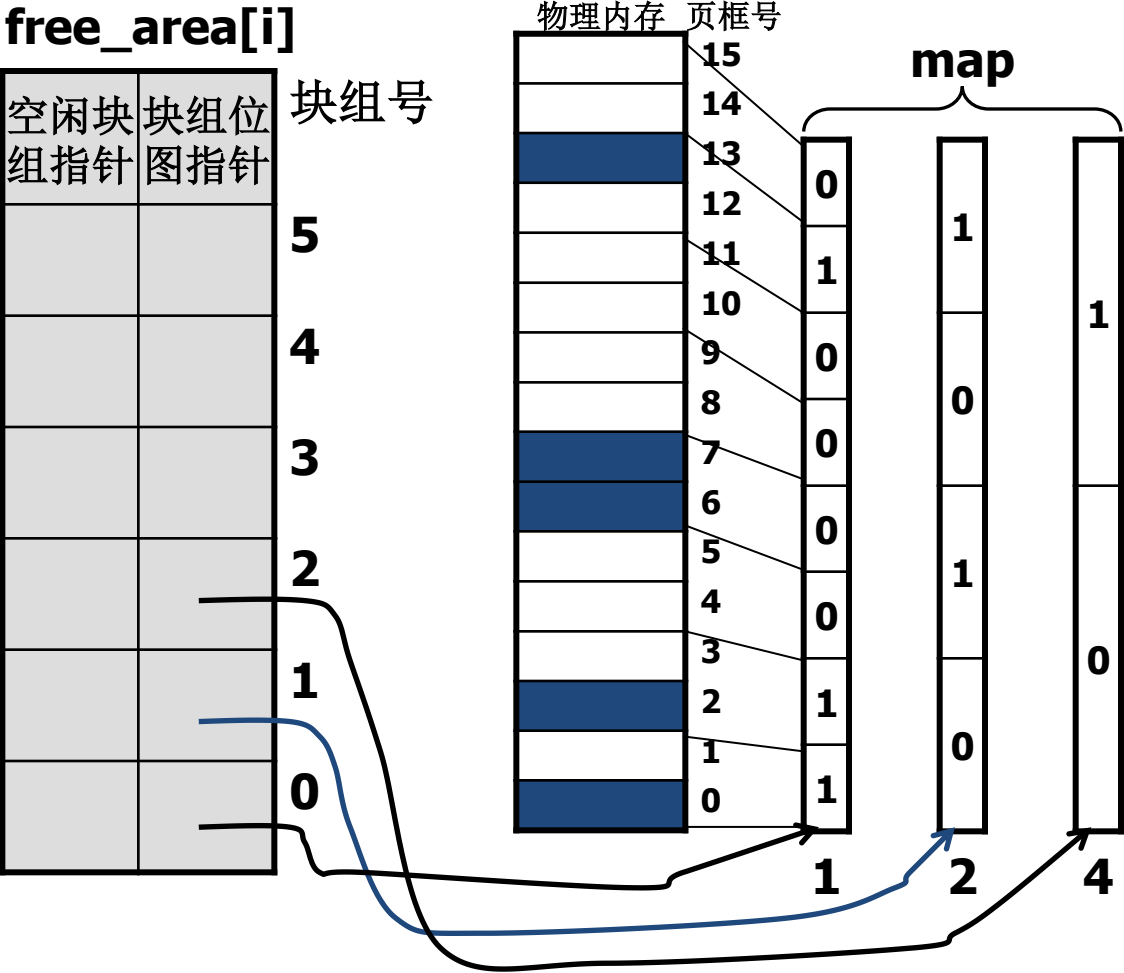
- 块组位图内容表示对应的一对块组伙伴的使用情况；
- 对于一对块组伙伴：
 - ① 若一个空闲，另一个全部或部分占用，则位图相应位置1；
 - ② 当两个都空闲，或都被全部或部分占用，则位图相应位置0。

■ 伙伴条件：

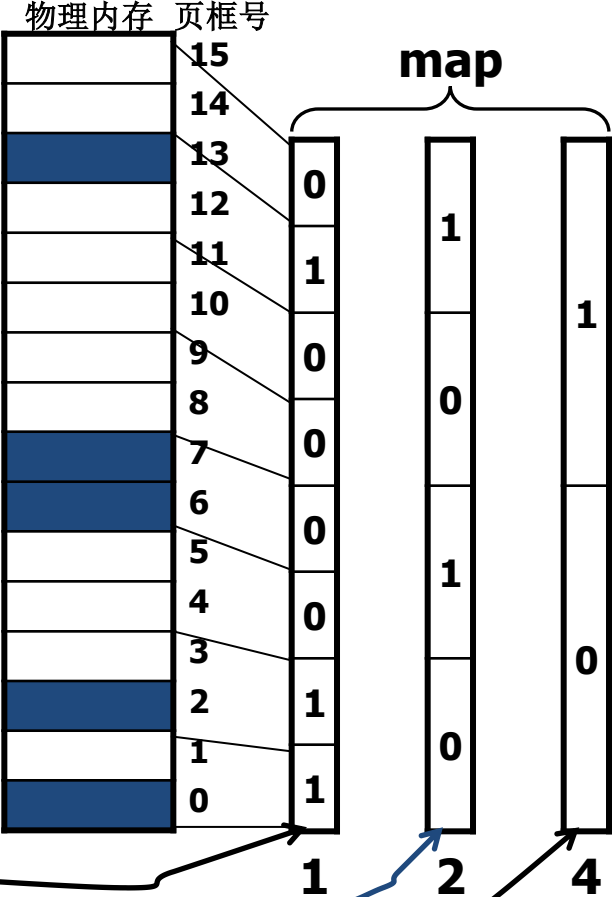
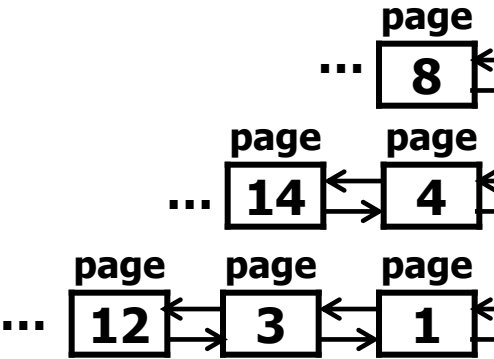
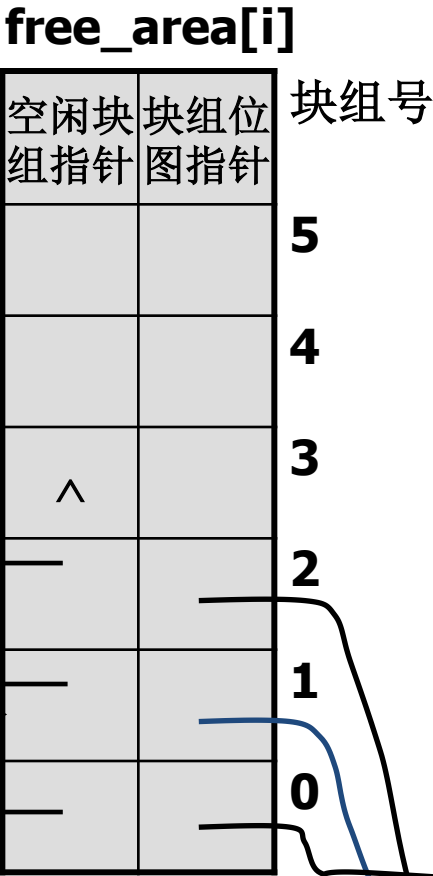
- ① 两个块大小相同，即具有相同的页框数 b ；
- ② 两个块的物理地址相连；
- ③ 如果以0作为页框的初始编号，那么位于后面块组的最后页框编号+1必须是 $2b$ 的整数倍。







■ 空闲区链表
组织结构:
假设6个块组



(2) Buddy heap algorithm

- 分配: 申请 fn 个页框

- ① 计算($2^{i-1} < fn \leq 2^i$), 找到相应的块组 j ;
- ② 在块组 j 的第一个空闲块分配 2^i 个页框;
- ③ 调整块组 j 的空闲块链表;
- ④ 如果块组 i 没有空闲块, 则向块组编号大的块组方向查找空闲块
- ⑤ 若 $2^j > 2^i$, 则把 $2^j - 2^i$ 个空闲页框加入到相应块组空闲链中;
(若 $2^j - 2^i$ 不是2的整数次幂, 则将其拆分成不同的整数次幂。)
- ⑥ 修改位图。

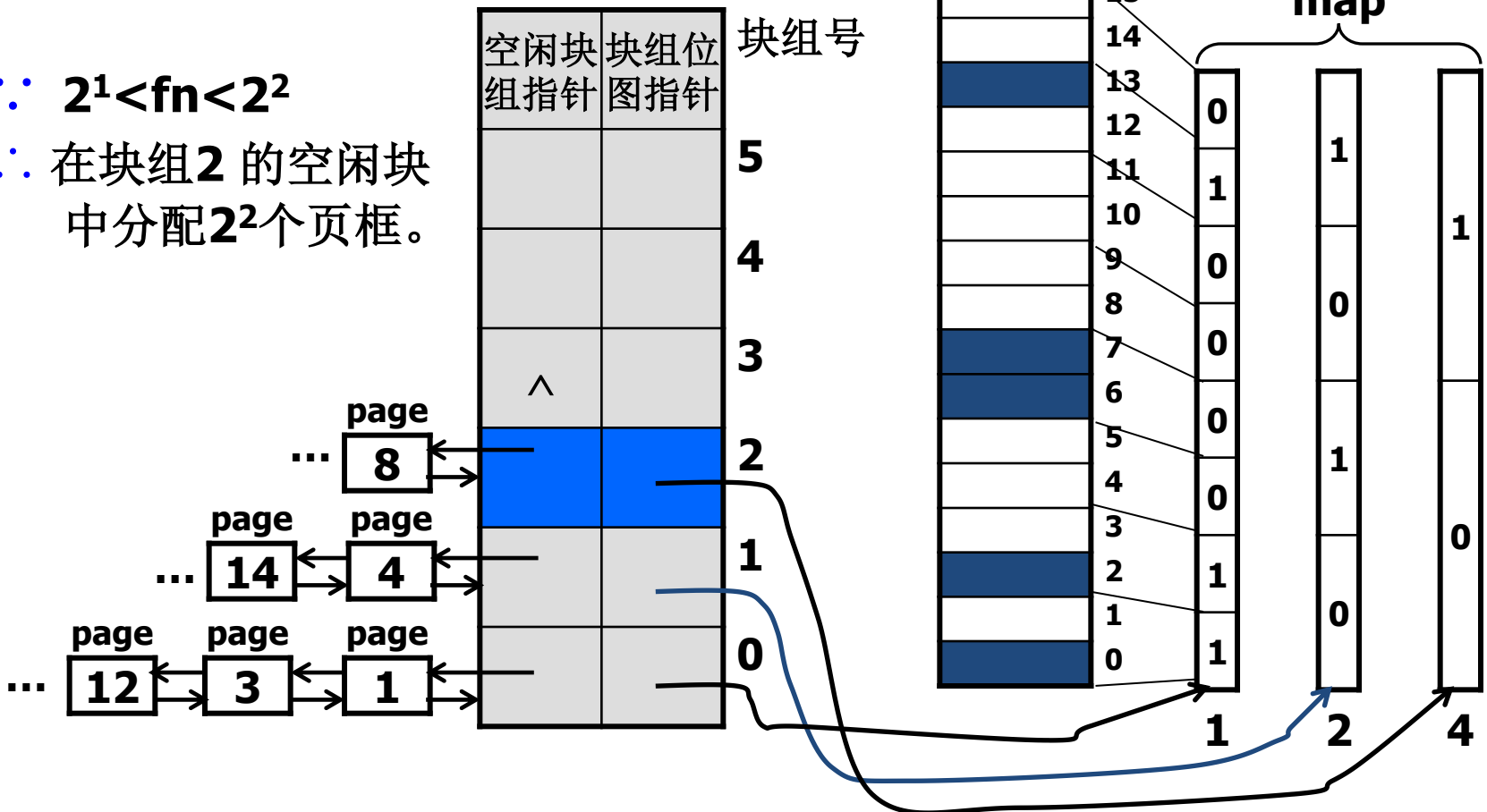
例如: 对于长度为128页的请求, 应该在第7组中取一块分配。如果第7组已空, 取第8组中的一块, 分配其中的128页, 并将剩余的128页加入第7组中。若第8组也空, 取第9组中的一块, 进行两次分割, 分配128页, 将剩余的128页和256页分别计入第7组和第8组。

(2) Buddy heap algorithm

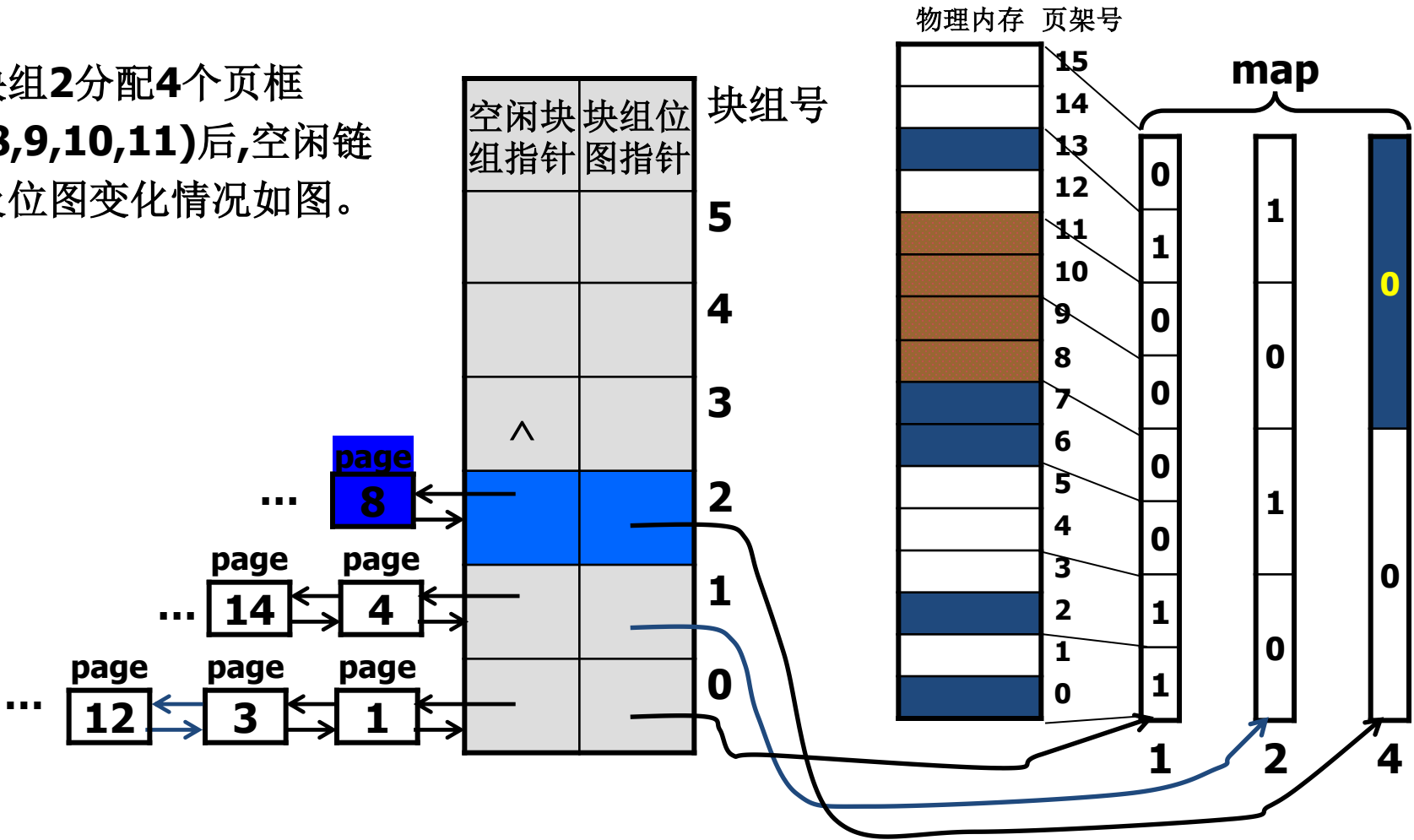
- 释放：释放 2^i 个页框
 - 释放的 2^i 个页框与相邻的空闲区按伙伴关系合并，
 - 即两个相邻的伙伴合并为一个大的空闲区；
 - 把得到的空闲区加入到不同块组的空闲链中；
- ③ 修改位图。

■ 分配例: 申请页框数fn=3

- ∵ $2^1 < fn < 2^2$
- ∴ 在块组2 的空闲块中分配 2^2 个页框。

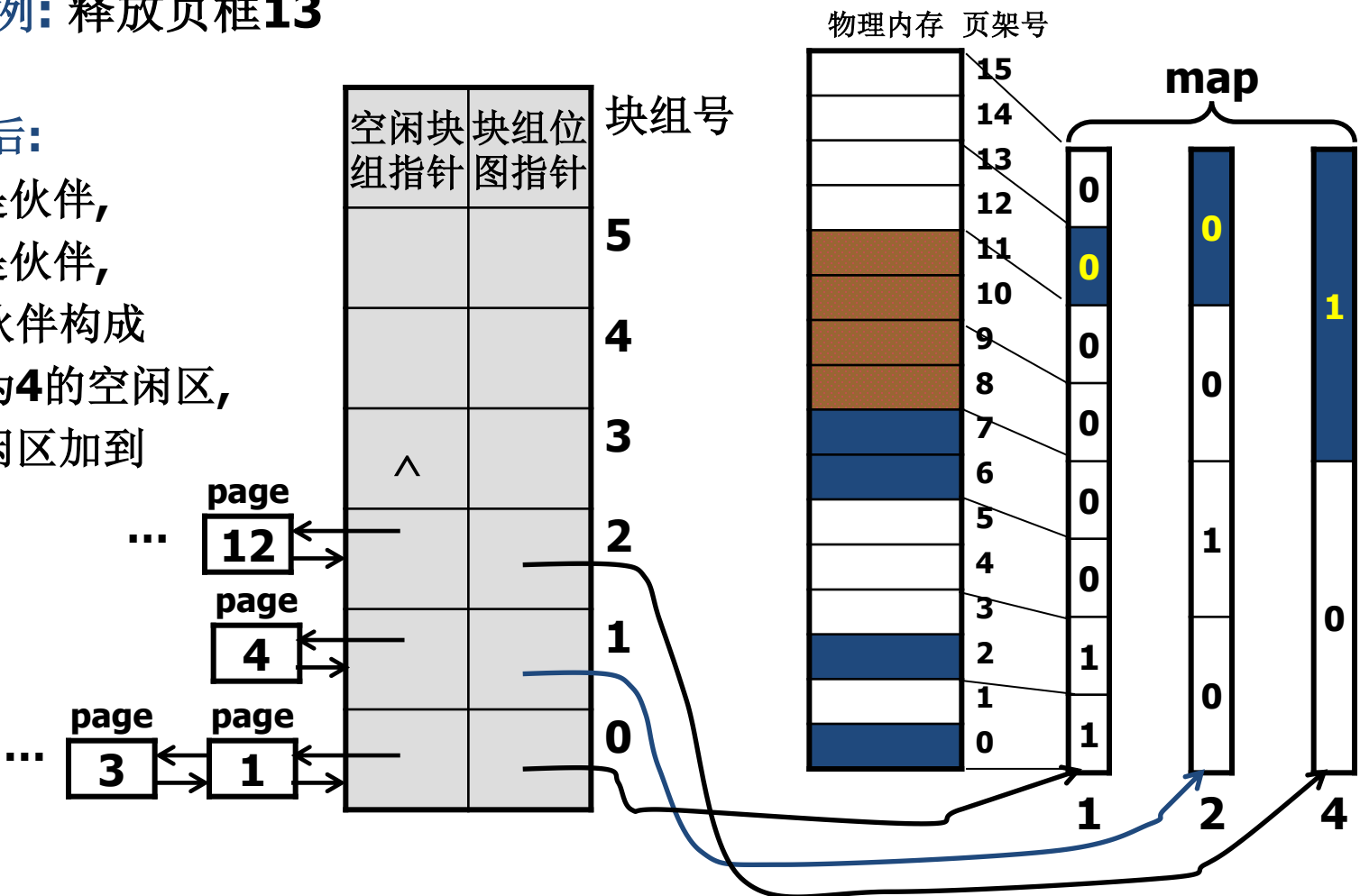


块组2分配4个页框
(8,9,10,11)后,空闲链
及位图变化情况如图。



■ 释放例: 释放页框13

释放**13**后:
12,13是伙伴,
14,15是伙伴,
这两个伙伴构成
页框数为**4**的空闲区,
将该空闲区加到
块组**2**。



■ Buddy heap algorithm:

● 问题: internal fragmentation.

例如: 申请17个页架, 由于 $2^4 < 17 \leq 2^5$,

按Buddy heap algorithm,

要在块组5的空闲区分配32个页框,

造成15个页框的浪费, 即internal fragmentation.

● 解决办法:

① second memory allocator

当实际申请页框数 $fn < 2^i$ 时

将 $2^i - fn$ 按2的整数次幂切分(carves slabs)

由second memory allocator单独管理

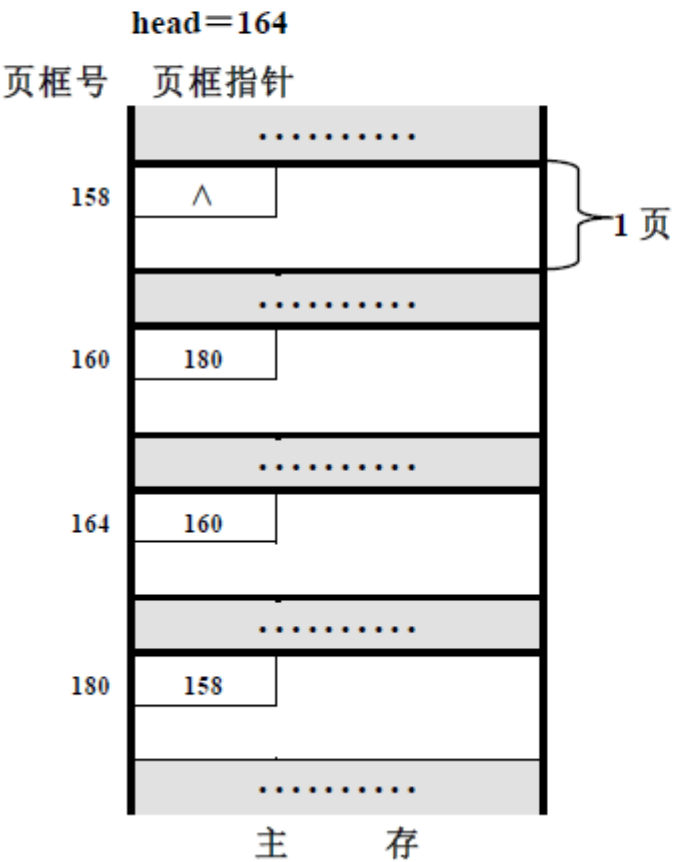
② third memory allocator

进程物理空间不要求连续时,

内存分配由third memory allocator完成。

例：2015年期末考试题

系统采用虚拟页式存储管理方式，页面大小为 2KB；内存页框分配采用静态策略，每个进程最多分配 3 个页框；主存空闲区管理采用空闲页面链结构，该链的头指针单元为 head；页置换采用改进的时钟算法，当有页面新装入内存时，页表“时钟”指针指向新装入页面表项的下一个在内存的表项。系统运行某时刻，假设“时钟”指针指向运行进程页表的第一个在内存的表项，内存空闲区情况及进程 P 和 Q 的页表如下：



页表中，进程 P 的页表表项

逻辑页号	页框号	引用位 r	修改位 m	内外标志
0	1BBH	1	0	0
1	0BCH	1	1	1
2	154H	1	1	0
3	1C0H	1	1	1

页表中，进程 Q 的页表表项

逻辑页号	页框号	引用位 r	修改位 m	内外标志
0	0ACH	1	1	1
1	1DAH	1	1	0

问 题： (1) 进程 P 依次执行：引用 15E7H；修改 3F0H；

写出 P 的页表内容以及逻辑地址 15E7H 和 3F0H 映射的物理地址；

(2) 在(1)的基础上，当进程 Q 执行 `exit()` 后，写出 head 及空闲页面链的内容。

要 求：页表中的页框号要填写十六进制数，主存映像的页框号及 head 值要写十进制数。

解：(1) (8分) 页面大小 $2KB=2*2^{10}=2^{11}$ ，则（物理地址或逻辑地址的）地址码中，低 11 位为页内偏移，高于 11 位的部分为物理页框号或逻辑页号；

- ◆ 逻辑地址 **15E7H** = 0001,0101,1110,0111B，高于 11 位是 2，所以，该逻辑地址访问 P 的逻辑页面 2，该页不在内存，缺页中断；P 可分配 3 个页框，而 P 当前只有两页在内存，可申请新页框。空闲页面链的头指针 **head** = 164 = **0A4H** (=1010, 0100B)，即分配给页面 2 的物理页框号为 0A4H，并且调整 **head** = 160；页面 2 装入后，“时钟”指针指向页面 3 的表项；P 执行“引用 **15E7H**”后，P 的页表如下：

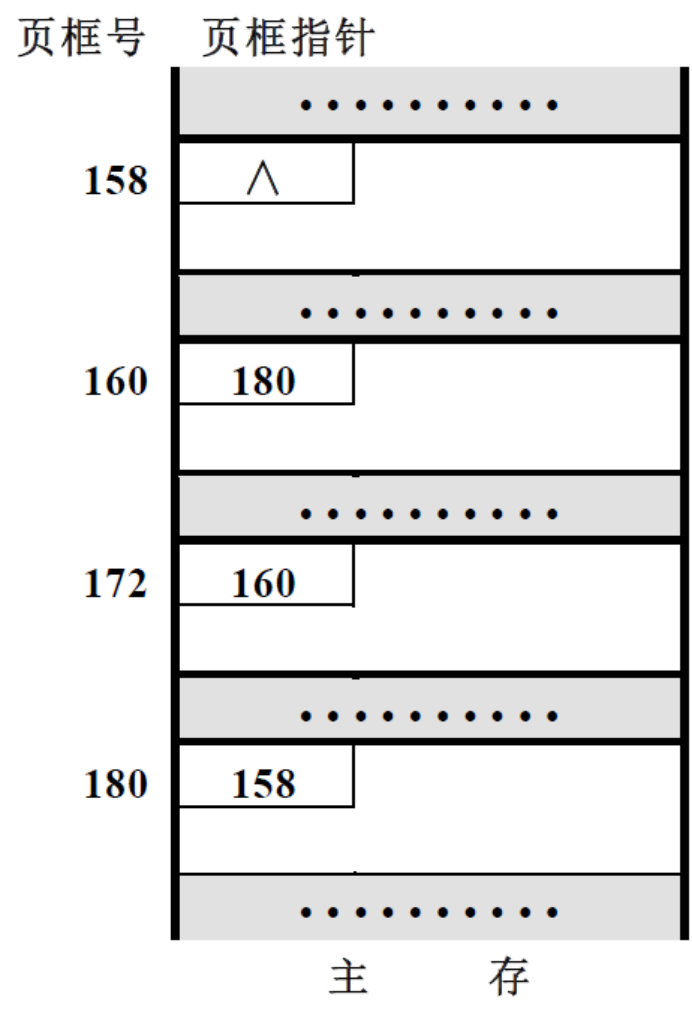
逻辑 页号	页框号	引用位 r	修改位 m	内外标志
0	1BBH	1	0	0
1	0BCH	1	1	1
2	0A4H	1	0	1
3	1C0H	1	1	1

- 逻辑地址 **15E7H** 映射的物理地址为：1010, 0100101,1110,0111B = **525E7H**

- ◆ **逻辑地址 3F0H = 0000,0011,1111,0000B**，高于 11 位是 0，
所以，该逻辑地址访问 P 的逻辑页面 0，该页不在内存，缺页中断；
P 已经分配 3 个页框，应淘汰页面。
按改进的时钟算法，此时“时钟”指针指向页面 3 的表项，应淘汰页面 2。
页面 2 分配的页框 0A4H = 0000,1010,1000B
- **逻辑地址 3F0H 对应的物理地址为：** 0000,1010,1000011,1111,0000B = 543F0H
- ◆ **P 修改逻辑地址 3F0H 并经改进的时钟算法扫描页表后，P 的页表如下：**

逻辑 页号	页框号	引用位 r	修改位 m	内外标志
0	0A4H	1	1	1
1	0BCH	0	1	1
2	0A4H	0	0	0
3	1C0H	0	1	1

(2) (2 分) 在 (1) 的基础上，当进程 Q 执行 `exit()` 后，要释放页框 0ACH=172 则 head=172；空闲页面链如下：



总 结

- 外存空间管理
 - 空闲块链
 - 空闲块表
 - 字位映像表
- 虚拟存储管理
 - 虚拟页式存储管理
 - 虚拟段式存储管理
 - 虚拟段页式存储管理

作业4

1 设某计算机主存有16个页框，内存分配和释放采用伙伴堆算法 (Buddy heap algorithm)。主存空闲区表的表项 $\text{free_area}[i]$ 的结构为：

空闲块组 <i>i</i> 的位图指针

空闲块组 <i>i</i> 的指针

空闲块组链的结点结构包括前、后结点指针和本空闲块组的首页框号3个数据域。设当前主存映像图及主存空闲区表 free_area 格式如下：

主存映像	页框号
占用	0
	1
占用	2
	3
	4
	5
占用	6
占用	7
	8
	9
	10
	11
	12
占用	13
	14
	15

块组 号 i	free_area	
	块组位 图指针	空闲块 组指针
0		
1		
2		
3		
4		

问题：

- (1) 根据主存映像图，写出伙伴堆算法的主存空闲区表`free_area`各表项的空闲块组链表和块组位图指向的块组位图内容的数据结构；
- (2) 基于 (1) 所做的数据结构图，设有长度为3页的内存申请，写出按伙伴堆算法分配页架后的主存映像图和伙伴堆算法数据结构图。

2 在页式存储管理系统中，现有J1、J2和J3共3个作业同驻内存。其中J2有4个页面，被分别装入内存的第3、4、6、8块中。假定页面和存储块的大小均为1024B，主存容量为10KB。

问 题：（1）写出J2的页表；

（2）当J2在CPU上运行时，执行到其地址空间第500号处遇到一条传送指令；MOV 2100, 3100, 请用地址变换图计算MOV指令中两个操作数的物理地址。

3 请求分页管理系统中，假设某进程的页表内容如下表所示：

页号	页框（Page Frame）号	有效位（存在位）
0	101H	1
1	--	0
2	254H	1

页面大小为**4KB**，一次内存的访问时间是**100ns**，一次快表（**TLB**）的访问时间是**10ns**，处理一次缺页的平均时间为**10⁸ns**（已含更新**TLB**和页表的时间），进程的驻留集大小固定为**2**，采用最近最少使用置换算法（**LRU**）和局部淘汰策略。假设①**TLB**初始为空；②地址转换时先访问**TLB**，若**TLB**未命中，再访问页表（忽略访问页表之后的**TLB**更新时间）；③有效位为**0**表示页面不在内存，产生缺页中断，缺页中断处理后，返回到产生缺页中断的指令处重新执行。设有虚地址访问序列**2362H**、**1565H**、**25A5H**，请问：

- （1）依次访问上述三个虚地址，各需多少时间？给出计算过程。
- （2）基于上述访问序列，虚地址**1565H**的物理地址是多少？请说明理由。